

GUIA DE ESTUDIO

TEORIA DE AUTOMATAS

Material:

Navarrete, I., Cardenas, M., Sanchez, D., Botía, J., Marín, R. Martínez, R. 2008. *Teoría de Autómatas y Lenguajes Formales*. Departamento de Ingeniería de la Información y las Comunicaciones. Universidad de Murcia

1. Defina "Autómata Finito".
 2. Defina "Autómata Finito Determinista (AFD)" y fundamente "determinista".
 3. Describa la ejecución de un AFD.
 4. Describa formas de representación de la "Función de Transición".
 5. Defina "Autómata Finito No Determinista (AFND)" y fundamente "no determinista".
 6. Ejemplifique en un AFND la "tabla de transición" y "diagrama de transición".
 7. Defina "Autómata Finito con λ -Transiciones (AFND- λ)".
 8. Defina "configuración de un AF".
 9. Defina "relación de cálculo de un paso".
 10. Defina "relación de cálculo" (clausura).
 11. Defina "clausura en n pasos".
 12. Defina "lenguaje aceptado por M".
 13. Defina "equivalencia entre AF".
 14. Defina la relación de aceptación de lenguajes entre un AFND y un AFD.
 15. Defina la relación de aceptación de lenguajes entre un AFND- λ y un AFND.
 16. Defina "Algoritmo de paso de AFND a AFD".
 17. Defina el Teorema de Análisis de Kleene.
 18. Defina el método que se propone para obtener una expresión regular α a partir de AF.
 19. Defina el Teorema de Síntesis de Kleene.
 20. Defina la relación de aceptación de lenguajes entre un AF y gramáticas regulares.
 21. Defina la relación entre lenguaje generado por una gramática regular y un AF.
 22. Defina "estado accesible de un AFD".
 23. Defina " estados equivalentes de un AFD". Ídem "estados distinguibles".
 24. Defina "Autómata Cociente".
 25. Defina la relación entre AFD y Autómata Cociente.
 26. Defina "Algoritmo de Minimización de un AFD".
-

CAPÍTULO 3: AUTÓMATAS FINITOS

Contenidos Teóricos

1. Arquitectura de un autómata finito (AF)
2. Autómatas finitos deterministas
3. Autómatas finitos no deterministas
4. Autómatas finitos con λ -transiciones
5. Lenguaje aceptado por un AF
6. Equivalencia entre autómatas finitos
7. Autómatas finitos, expresiones regulares y gramáticas regulares
8. Minimización de un AFD
9. Aplicaciones: análisis léxico

1. Arquitectura de un autómata finito (AF)

Un autómata finito es una estructura matemática que representa un sistema o máquina abstracta cuya arquitectura puede verse en la figura 3.1

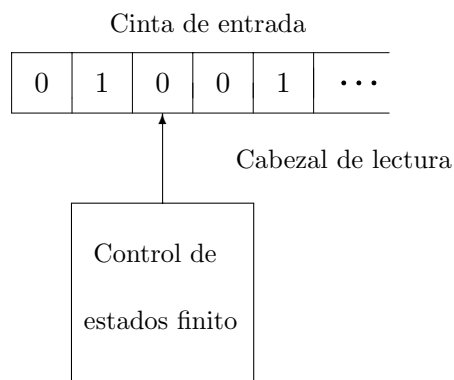


Figura 3.1: Arquitectura de un AF

La *cinta de entrada* (que se extiende infinitamente hacia la derecha) está dividida en celdas, cada una de las cuales es capaz de almacenar un sólo símbolo de un cierto alfabeto. La máquina es capaz de leer los símbolos de esta cinta de izquierda a derecha por medio de un *cabezal de lectura*. Cada vez que se lee un símbolo, el cabezal de lectura se mueve a la siguiente celda a la derecha y la máquina efectúa un cambio de estado o *transición*. Esta transición está determinada por

el *mecanismo de control* (que contiene un número finito de estados), programado para conocer cual debe ser el nuevo estado, que dependerá de la combinación del estado actual y el símbolo de entrada leído.

Los autómatas finitos pueden considerarse como mecanismos *aceptadores* o reconocedores de palabras. De manera informal decimos que un autómata finito aceptará una palabra de entrada si, comenzando por un estado especial llamado *estado inicial* y estando la cabeza de lectura apuntando al primer símbolo de la cadena, la máquina alcanza un *estado final* o de aceptación después de leer el último símbolo de la cadena.

2. Autómatas finitos deterministas

Un AUTÓMATA FINITO DETERMINISTA (*AFD*) se define como una quintupla $M = (Q, V, \delta, q_0, F)$, donde:

Q es un *conjunto finito de estados*
 V es el *alfabeto de entrada*
 q_0 es el *estado inicial*
 $F \subseteq Q$ es el *conjunto de estados finales*
 $\delta : Q \times V \longrightarrow Q$ es la *función de transición*

El nombre “determinista” viene de la forma en que está definida la función de transición: si en un instante t la máquina está en el estado q y lee el símbolo a entonces, en el instante siguiente $t + 1$ la máquina cambia de estado y sabemos con seguridad cual es el estado al que cambia, que es precisamente $\delta(q, a)$.

El *AFD* es **inicializado** con una palabra de entrada w como sigue:

1. w se coloca en la cinta de entrada, con un símbolo en cada celda
2. el cabezal de lectura se apunta al símbolo más a la izquierda de w
3. el *estado actual* pasa a ser q_0

Una vez que se ha inicializado el *AFD*, comienza su “ejecución” sobre la palabra de entrada. Como cualquier computador tiene un **ciclo de ejecución básico**:

1. se lee el *símbolo actual*, que es el apuntado por el cabezal de lectura. Si el cabezal apunta a una celda vacía entonces el *AFD* termina su ejecución, *aceptando* la palabra en caso de que el estado actual sea final y *rechazando* la palabra en caso contrario. Esto ocurre cuando se ha leído toda la palabra de entrada, y se produce una situación similar a tener una condición “fin de fichero” en la ejecución de un programa
2. se calcula el *estado siguiente* a partir del estado actual y del símbolo actual según la función de transición, esto es, $\delta(\text{estado actual}, \text{símbolo actual}) = \text{estado siguiente}$
3. el cabezal de lectura se mueve una celda a la derecha
4. el estado siguiente pasa a ser el estado actual y vuelve al paso 1

La función de transición de un *AFD* se puede **representar** de dos formas: mediante una tabla de transición o mediante un diagrama de transición.

Tabla de transición	Cada fila corresponde a un estado $q \in Q$ El estado inicial se precede del símbolo $\rightarrow$ Cada estado final se precede del símbolo $\#$ Cada columna corresponde a un símbolo de entrada $a \in V$ En la posición (q, a) está el estado que determine $\delta(q, a)$
Diagrama de transición	Los nodos se etiquetan con los estados El estado inicial tiene un arco entrante no etiquetado Los estados finales están rodeados de un doble círculo Habrá un arco etiquetado con a desde el nodo q_i al q_j si $\delta(q_i, a) = q_j$

Ejemplo 3.1 Supongamos que tenemos el autómata finito determinista dado por

$$M = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

donde la función $\delta : \{q_0, q_1, q_2\} \times \{0, 1\} \longrightarrow \{q_0, q_1, q_2\}$ viene dada por

$$\begin{array}{ll} \delta(q_0, 0) = q_0 & \delta(q_0, 1) = q_1 \\ \delta(q_1, 0) = q_0 & \delta(q_1, 1) = q_2 \\ \delta(q_2, 0) = q_2 & \delta(q_2, 1) = q_1 \end{array}$$

La tabla de transición correspondiente a este autómata será:

δ	0	1
$\rightarrow q_0$	q_0	q_1
$\# q_1$	q_0	q_2
q_2	q_2	q_1

y el diagrama de transición correspondiente se muestra en la figura 3.2.

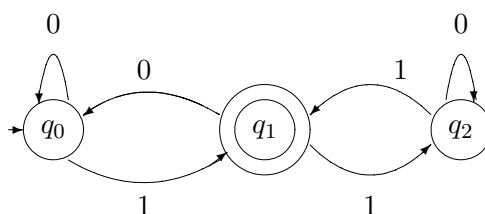


Figura 3.2: Diagrama de transición del ejemplo 3.1

Nota El diagrama de transición de un *AFD* tiene por cada nodo un sólo arco etiquetado con cada uno de los símbolos del alfabeto. Algunos autores consideran que la función de transición puede ser *parcial*, es decir, no estar definida para algún $\delta(q, a)$. En ese caso se dice que el *AFD* es *incompleto*, y en el diagrama de transición faltarían los arcos correspondientes a los casos no definidos de la función de transición. Nosotros consideraremos que los *AFDs* son *completos*.

3. Autómatas finitos no deterministas

Un AUTÓMATA FINITO NO DETERMINISTA (*AFND*) es una quintupla $M = (Q, V, \Delta, q_0, F)$ donde todos los componentes son como en los *AFDs*, excepto la función de transición que se define ahora como:

$$\Delta : Q \times V \longrightarrow \mathcal{P}(Q)$$

donde $\mathcal{P}(Q)$ denota el conjunto de las partes de Q (o conjunto potencia 2^Q).

El hecho de que el codominio de la función de transición sea $\mathcal{P}(Q)$ es lo que añade esta característica de “no determinismo”: a partir del estado actual y del símbolo actual de entrada no se puede determinar de forma exacta cuál será el estado siguiente. Por ejemplo, podemos tener $\Delta(q, a) = \{q_1, q_2, \dots, q_m\}$ y esto indica que dado el estado actual q y el símbolo de entrada a , el estado siguiente puede ser cualquier estado entre q_1 y q_m . También puede darse el caso de que $\Delta(q, a) = \emptyset$, lo que indica que el estado siguiente no está definido.

Intuitivamente, un *AFND* acepta una palabra de entrada w siempre que sea posible comenzar por el estado inicial y que exista una secuencia de transiciones que nos lleven a consumir la palabra y acabe el autómata en un estado final. Puede que tengamos otras secuencias de transiciones que no acaben en estado final, pero basta que exista una que acabe en estado final para que la palabra sea aceptada.

Los *AFND* también se representan mediante tablas o diagramas de transición. En el diagrama de transición, el no determinismo se descubre porque hay algún nodo del que parten dos o más arcos etiquetados con el mismo símbolo del alfabeto, o falta algún arco para algún símbolo del alfabeto. En la figura 3.3 podemos ver un ejemplo de tabla y diagrama de transición de un *AFND*.

δ	a	b
$\rightarrow q_0$	$\{q_0, q_3\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\{q_2\}$
$\# q_2$	$\{q_2\}$	$\{q_2\}$
q_3	$\{q_4\}$	$\emptyset$
$\# q_4$	$\{q_4\}$	$\{q_4\}$

Tabla de transición

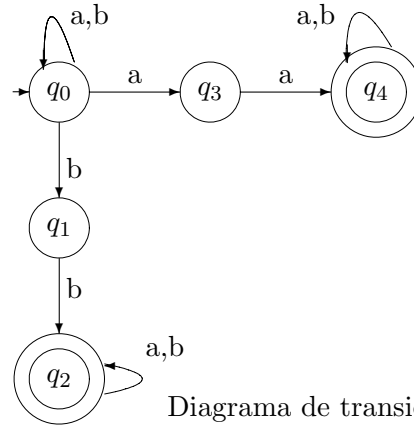


Diagrama de transición

Figura 3.3: Ejemplo de *AFND*

4. Autómatas finitos con λ -transiciones

Un AUTÓMATA FINITO CON λ -TRANSICIONES (*AFND- λ*) es básicamente un *AFND* al que se le permite cambiar de estado sin necesidad de consumir o leer un símbolo de la entrada. Por eso la función de transición de un *AFND- λ* se define

$$\Delta : Q \times (V \cup \{\lambda\}) \longrightarrow \mathcal{P}(Q)$$

La tabla de transición de un $AFND-\lambda$ es como la de un $AFND$ excepto que se le añade una columna correspondiente a λ , de forma que en la posición $T[(q, \lambda)]$ estará el conjunto de estados que determine $\Delta(q, \lambda)$.

Ejemplo 3.2 Supongamos que tenemos un $AFND-\lambda$ cuyo diagrama de transición corresponde al de la figura 3.4. Entonces si el autómata está en el estado q_1 en un cierto instante y el símbolo actual es b , en el instante siguiente, el autómata puede decidir de forma no determinista entre “leer el símbolo b y cambiar al estado q_4 ”, o bien, “cambiar al estado q_2 sin mover el cabezal de lectura”. Además, el conjunto de cadenas que es capaz de aceptar este autómata es $\{b, bb, bbb\}$.

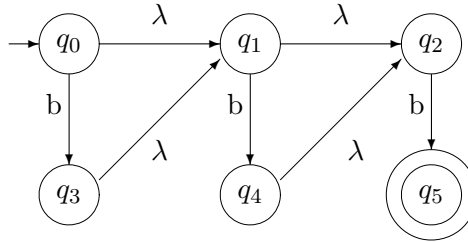


Figura 3.4: Ejemplo de $AFND-\lambda$

5. Lenguaje aceptado por un AF

Un autómata finito sirve para reconocer cierto tipo de lenguajes. Antes de definir formalmente el concepto de lenguaje aceptado por un AF necesitamos definir los conceptos de *configuración* y *cálculo* en un autómata finito.

La configuración de un autómata finito (sin importar el tipo) en cierto instante viene dada por el estado del autómata en ese instante y por la porción de cadena de entrada que le queda por leer o procesar. La porción de cadena leída hasta llegar al estado actual no tiene influencia en el comportamiento futuro de la máquina. En este sentido podemos decir que un AF es una máquina sin memoria externa; son los estados los que resumen de alguna forma la información procesada.

Formalmente una CONFIGURACIÓN de un AF es un elemento $(q, w) \in (Q \times V^*)$. Algunos tipos de configuraciones especiales son:

- *Configuración inicial*: (q_0, w) , donde q_0 es el estado inicial y w la palabra de entrada.
- *Configuración de parada*: cualquier configuración en la que el autómata puede parar su ejecución, bien porque se haya procesado toda la entrada o bien porque se haya llegado a una situación donde no es aplicable ninguna transición.
- *Configuración de aceptación*: (q_F, λ) , donde q_F es un estado final del autómata. Una vez alcanzada esta configuración el autómata puede aceptar la palabra.

Si consideramos el conjunto de las configuraciones de un autómata finito, podemos definir una relación binaria $\vdash \subseteq (Q \times V^*) \times (Q \times V^*)$ que llamaremos *relación de cálculo en un paso*. Intuitivamente si dos configuraciones C_i y C_j están relacionadas mediante la relación $\vdash$ y lo notamos como $C_i \vdash C_j$, quiere decir que podemos pasar de la configuración C_i a la C_j aplicando una sola transición y diremos que “la configuración C_i alcanza en un paso la configuración C_j ”.

Para definir formalmente la relación de cálculo en un paso $\vdash$, distinguiremos tres casos correspondientes a los tres tipos de autómatas que hemos visto:

- Si tenemos un *AFD*, la relación de cálculo en un paso se define de la siguiente forma:

$$(q, w) \vdash (q', w') \Leftrightarrow \begin{cases} w = aw', \text{ donde } a \in V \\ q' = \delta(q, a) \end{cases}$$

- Si tenemos un *AFND*, la relación de cálculo en un paso la se define:

$$(q, w) \vdash (q', w') \Leftrightarrow \begin{cases} w = aw', \text{ donde } a \in V \\ q' \in \Delta(q, a) \end{cases}$$

- Si tenemos un *AFND- λ* , la relación de cálculo en un paso se define:

$$(q, w) \vdash (q', w') \Leftrightarrow \begin{cases} w = \sigma w', \text{ donde } \sigma \in V \cup \{\lambda\} \\ q' \in \Delta(q, \sigma) \end{cases}$$

Cuando queramos distinguir el autómata M al que refiere la relación, se usará $\vdash_M$.

La clausura reflexiva y transitiva de la relación $\vdash$ es otra relación binaria $\vdash^* \subseteq (Q \times V^*) \times (Q \times V^*)$, que llamaremos *relación de cálculo*. Diremos que la “configuración C_i alcanza (en cero o más pasos) la configuración C_j ”, y lo notamos como $C_i \vdash^* C_j$, si se cumple una de las dos condiciones siguientes:

1. $C_i = C_j$, o bien,
2. $\exists C_0, C_1, \dots, C_n$, tal que $C_0 = C_i$, $C_n = C_j$, y $\forall 0 \leq k \leq n-1$ se cumple que $C_k \vdash C_{k+1}$

A una secuencia del tipo $C_0 \vdash C_1 \vdash \dots \vdash C_n$ la llamaremos *cálculo en n pasos*, abreviadamente $C_0 \vdash_{n \text{ pasos}}^* C_n$.

Ejemplo 3.3 Considerando el *AFD* de la figura 3.2 podemos decir que $(q_0, 01) \vdash (q_0, 1)$, $(q_0, 1) \vdash (q_1, \lambda)$ y por tanto $(q_0, 01) \vdash^* (q_1, \lambda)$. También $(q_1, 101) \vdash (q_2, 01)$ y en varios pasos $(q_2, 0011) \vdash^* (q_1, 1)$.

Por otra parte para el *AFND* de la figura 3.3 tenemos, por ejemplo, que $(q_0, abb) \vdash (q_0, bb)$ y también $(q_0, abb) \vdash (q_3, bb)$. Al ser el autómata no determinista vemos que a partir de una misma configuración, en este caso (q_0, abb) , se puede llegar en un paso de cálculo a dos o más configuraciones distintas. Esta situación no puede producirse en un *AFD*.

Para el *AFND- λ* de la figura 3.4 el cálculo $(q_1, bb) \vdash (q_2, bb)$ es un ejemplo donde se produce una transición que implica un cambio de estado sin consumir símbolos de entrada. Esto es posible porque $q_2 \in \Delta(q_1, \lambda)$.

Si tenemos un autómata finito $M = (Q, V, \delta, q_0, F)$, se define el *lenguaje aceptado por M* y lo notamos $L(M)$, como:

$$L(M) = \{w \in V^* \mid (q_0, w) \vdash^* (q_F, \lambda) \text{ donde } q_F \in F\}$$

Es decir, una palabra w será aceptada por el autómata M , si partiendo de la configuración inicial con w en la cinta de entrada, el autómata es capaz de alcanzar una configuración de aceptación. Dependiendo del tipo de autómata de que se trate, $\vdash^*$ hará referencia a la clausura reflexiva y transitiva de la relación $\vdash$ en un *AFD*, en un *AFND* o en un *AF* con λ -transiciones. En un autómata finito determinista, el hecho de que una palabra w sea aceptada por el autómata nos asegura que existe un único camino en el diagrama de transición que nos lleva del nodo etiquetado con el estado inicial al nodo etiquetado con el estado final y cada arco que se recorre

en este camino está etiquetado con un símbolo de la palabra. Podríamos simular la ejecución de un autómata finito determinista mediante un programa que codifique la función de transición y simule los cambios de estado. Si $|w| = n$ entonces el programa puede determinar si la palabra es aceptada o no en $O(n)$.

En el caso de un *AFND* o un *AFND-λ* no podemos asegurar que exista un único camino en el diagrama que nos lleve del estado inicial a un estado final consumiendo los símbolos de la palabra. Incluso puede que para una palabra $w \in L(M)$ podamos tener una camino que no acabe en estado final o que llegue a un estado desde el que no se pueda seguir leyendo símbolos. Esto es debido al no determinismo, que hace que los cálculos en estos autómatas no estén perfectamente determinados. Si quisiéramos simular un autómata no determinista para decidir si una palabra es aceptada o no, tendríamos que usar alguna técnica de retroceso o *backtracking* para explorar distintas posibilidades hasta encontrar un cálculo correcto que reconozca la palabra o determinar que la palabra no es aceptada si se han explorado todos los posibles cálculos y ninguno de ellos conduce a un estado final. Esto nos llevaría a un algoritmo de tiempo exponencial para reconocer una palabra. De ahí que a efectos prácticos, como en la construcción de analizadores léxicos o reconocimiento de patrones en un texto, lo deseable es tener un autómata finito determinista. Afortunadamente siempre es posible pasar de un *AF* no determinista a un *AF* determinista, como veremos en la siguiente sección.

Ejemplo 3.4 Recordemos los *AFs* ya vistos y veamos ahora cuál es el lenguaje aceptado por ellos. El diagrama de la figura 3.3 correspondiente a un *AFND* permite ver de forma intuitiva que $L(M)$ es el lenguaje descrito por la expresión regular $(a + b)^*(aa + bb)(a + b)^*$ que consiste en aquellas cadenas sobre el alfabeto $V = \{a, b\}$ que contienen al menos una ocurrencia de la subcadena aa ó bb . Por ejemplo, la cadena abb es aceptada, ya que tenemos el cálculo:

$$(q_0, abb) \vdash (q_0, bb) \vdash (q_1, b) \vdash (q_2, \lambda), \quad y \quad q_2 \in F$$

Sin embargo podemos tener otro cálculo que no conduce a estado final:

$$(q_0, abb) \vdash (q_0, bb) \vdash (q_0, b) \vdash (q_1, \lambda), \quad q_1 \notin F$$

e incluso un cálculo que no llega a consumir la palabra:

$$(q_0, abb) \vdash (q_3, bb) \vdash \text{(y no puede seguir)}$$

A partir del diagrama del *AFD* de la figura 3.2 no es tan sencillo ver cuál es el lenguaje aceptado. Pero, según veremos en la sección 7, hay un método exacto para encontrar este lenguaje. En este caso el lenguaje aceptado es el descrito por la expresión regular $(0 + 1(10^*1)^*)^* 1(10^*1)^*$

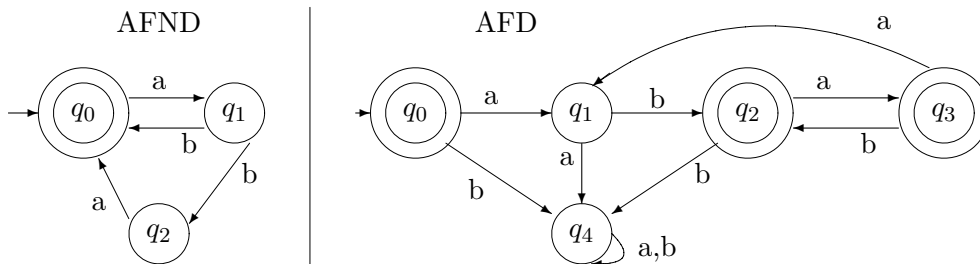


Figura 3.5: *AFs* que aceptan $L(\alpha)$ donde $\alpha = (ab + aba)^*$

6. Equivalencia entre autómatas finitos

Decimos que dos autómatas finitos M y M' son *equivalentes* si y sólo si aceptan el mismo lenguaje, esto es, $L(M) = L(M')$. Veremos ahora que, en contra de lo que parece, los autómatas no deterministas (con o sin λ -transiciones) son igual de potentes que los autómatas finitos deterministas, en el sentido de que son capaces de reconocer los mismos lenguajes: los lenguajes regulares.

Ya hemos dicho que a efectos de simular la ejecución de un AF con un programa conviene que el autómata sea determinista para poder reconocer las palabras en tiempo polinomial. ¿Qué sentido tienen entonces los autómatas no deterministas? Desde el punto de vista teórico son interesantes porque permiten modelizar algoritmos de búsqueda y retroceso y también son de gran utilidad, sobre todo los $AFND-\lambda$, para demostrar algunos teoremas sobre autómatas y lenguajes formales. Otra ventaja que supone el uso de autómatas no deterministas es que a veces resulta más intuitivo y sencillo diseñar un autómata no determinista para reconocer un determinado lenguaje, que pensar directamente en el autómata determinista. Un ejemplo lo tenemos en la figura 3.5. Se puede observar que es más sencillo averiguar el lenguaje aceptado a partir del $AFND$ que del AFD .

Para demostrar formalmente que los tres tipos de autómatas vistos aceptan la misma clase de lenguajes, vamos a ver dos teoremas, uno de ellos establece la equivalencia entre los modelos de AFD y $AFND$ y el otro teorema demuestra la equivalencia entre $AFND$ y $AFND-\lambda$. Las demostraciones de estos teoremas tienen una parte constructiva que muestra un método algorítmico para pasar de un tipo de autómata a otro, y una parte inductiva que prueba la validez de dicho método.

Teorema 3.1 *Un lenguaje es aceptado por un $AFND$ si y sólo si es aceptado por un AFD .*

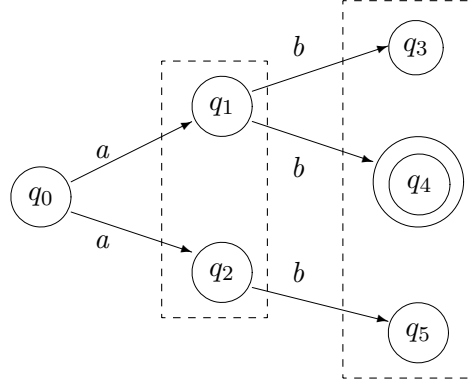
Dem.- Esta claro que un AFD se puede considerar como un caso particular de $AFND$ donde $|\Delta(q, a)| = 1, \forall q \in Q, a \in V$. Por tanto si un lenguaje L es aceptado por un AFD también será aceptado por un $AFND$.

Supongamos que un lenguaje L es aceptado por un autómata finito no determinista $M_N = (Q, V, \Delta, q_0, F)$. Vamos a construir a partir de él un autómata finito determinista $M_D = (Q', V, \delta, q'_0, F')$ que tiene las siguientes componentes:

$$\begin{aligned} Q' &= \mathcal{P}(Q) \\ q'_0 &= \{q_0\} \\ F' &= \{S \in \mathcal{P}(Q) \mid S \cap F \neq \emptyset\} \\ \forall S \in Q', a \in V \text{ se define } \delta(S, a) &= \bigcup_{q \in S} \Delta(q, a) = \{p \in Q \mid p \in \Delta(q, a) \wedge q \in S\} \end{aligned}$$

Tenemos pues, que el conjunto de estados del AFD así construido está formado por estados que a su vez son conjuntos de estados del $AFND$. La idea es la siguiente: cuando el $AFND$ lee una cadena w , no sabemos exactamente en que estado quedará el autómata, pero podemos decir que estará en un estado de un posible conjunto de estados, por ejemplo, $\{q_i, q_j, \dots, q_k\}$. Un AFD equivalente después de leer la misma entrada quedará en un estado que está perfectamente determinado, y para que el autómata AFD sea equivalente haremos que este estado corresponda al conjunto de estados $\{q_i, q_j, \dots, q_k\}$. Por eso, si originalmente tenemos $|Q|$ estados en el $AFND$, entonces el AFD tendrá como mucho $2^{|Q|}$ estados.

Ejemplo 3.5 *Supongamos que tenemos el $AFND$ siguiente:*



Al leer la palabra $w = ab$ se pueden dar los siguientes cálculos:

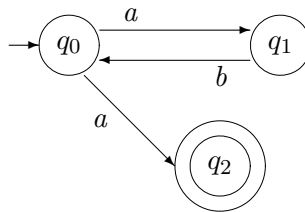
$$\begin{array}{c}
 (q_0, ab) \vdash_{M_N} \nearrow (q_1, b) \vdash_{M_N} \nearrow (q_3, \lambda) \\
 \searrow (q_4, \lambda) \\
 \searrow (q_2, b) \vdash_{M_N} (q_5, \lambda)
 \end{array}$$

El autómata acepta la palabra porque uno de los posibles cálculos conduce a un estado final. Consecuentemente, el AFD deberá reconocer la palabra. De hecho es así y el único cálculo que llega a consumir la palabra y acabar en estado final es:

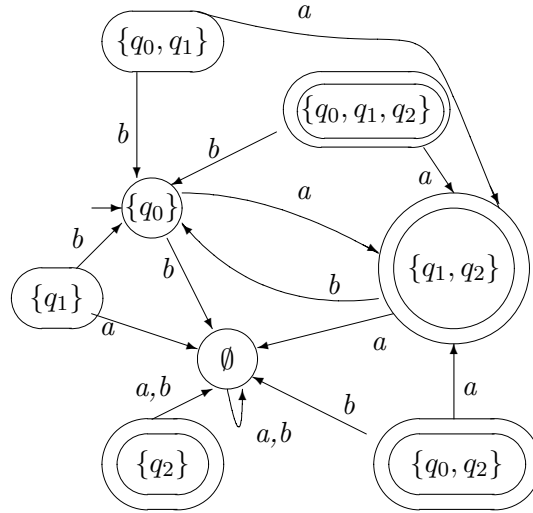
$$(\{q_0\}, ab) \vdash_{M_D} (\{q_1, q_2\}, b) \vdash_{M_D} (\{q_3, q_4, q_5\}, \lambda)$$

Para acabar de demostrar el teorema necesitamos probar que $L(M_D) = L(M_N)$, o lo que es lo mismo, que $w \in L(M_D) \Leftrightarrow w \in L(M_N)$, $\forall w \in V^*$. Lo dejamos para la sección de aplicaciones. ■

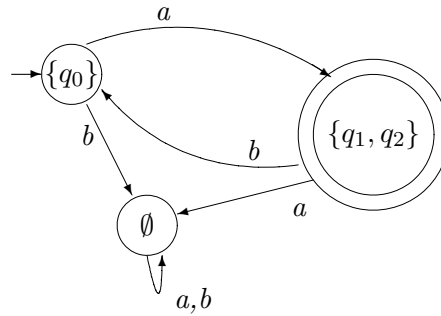
Ejemplo 3.6 Dado el lenguaje descrito por la expresión regular $(ab)^*a$, un AFND que acepta dicho lenguaje es el siguiente:



Si aplicamos el método del teorema 3.1 anterior se obtiene un AFD equivalente con 8 estados, correspondientes a los 8 subconjuntos de estados del AFND:



En el diagrama anterior podemos ver que hay estados (los enmarcados en un recuadro oval) que son inaccesibles, es decir, no se puede llegar hasta ellos desde el estado inicial. Por tanto podrían eliminarse del AFD sin que ello afecte al lenguaje aceptado, obteniéndose el siguiente AFD simplificado:



El método del teorema 3.1 considera todos los posibles elementos de $\mathcal{P}(Q)$, pero ya hemos visto que no siempre son todos necesarios. En la figura 3.6 se presenta un **algoritmo** que calcula un AFD a partir de un AFND siguiendo la misma idea que antes, pero *sin generar estados inaccesibles*.

Teorema 3.2 Un lenguaje es aceptado por un AFND- λ si y sólo si es aceptado por un AFND.

Dem.- Es obvio que un AFND puede considerarse un caso restringido de AFND- λ , donde $\Delta(q, \lambda) = \emptyset, \forall q \in Q$. Por tanto si un lenguaje L es aceptado por un AFND también será aceptado por un AFND- λ .

Se define la λ -clausura de un estado q en una AFND- λ como:

$$\lambda\text{-clau}(q) = \{p \in Q \mid (q, \lambda) \vdash^* (p, \lambda)\}$$

Esto es, la λ -clausura de un estado nos da el conjunto de estados que se pueden alcanzar siguiendo todos los caminos en el diagrama de transición que parten del nodo q y sólo pasan por arcos etiquetados con λ . También se tiene, por definición, que $q \in \lambda\text{-clau}(q)$. Esta definición

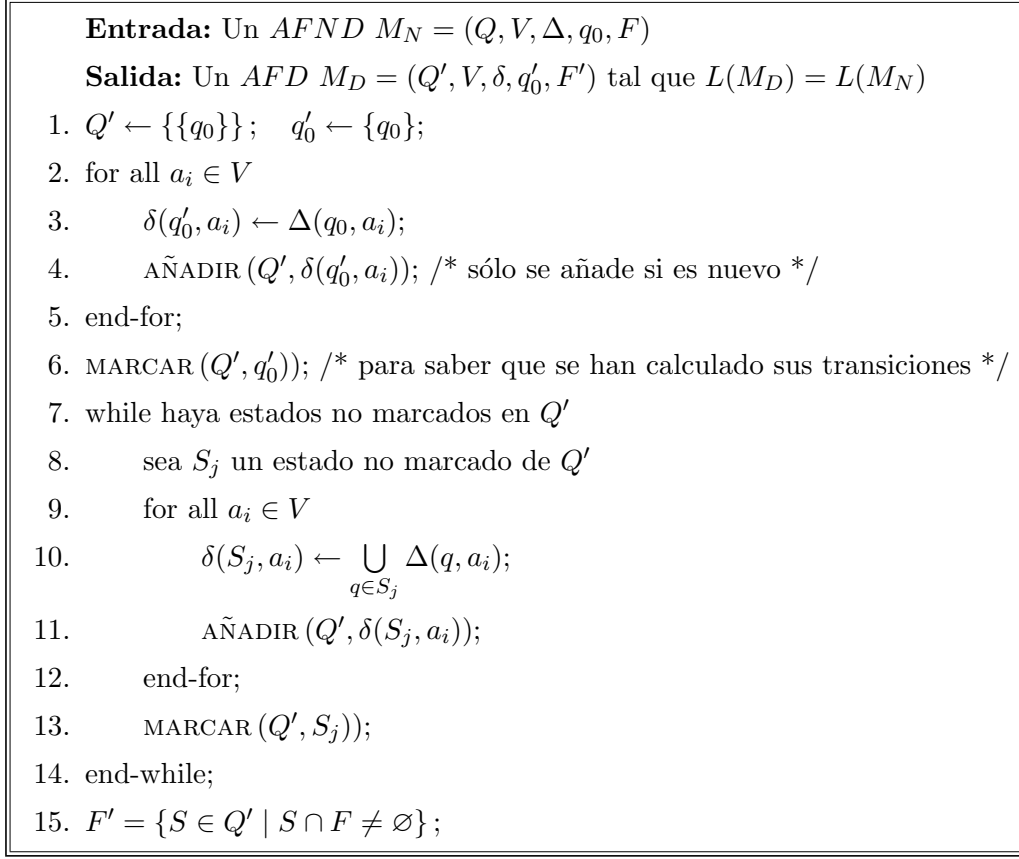


Figura 3.6: Algoritmo de paso de *AFND* a *AFD*

se puede extender y se define la λ -clausura de un conjunto de estados S como:

$$\lambda\text{-clau}(S) = \bigcup_{q \in S} \lambda\text{-clau}(q)$$

Cuando un *AFND*- λ parte de un estado q y lee un símbolo a , el autómatas podrá alcanzar un conjunto de estados, que son los estados a los que se puede llegar desde q siguiendo todos los caminos con el siguiente procedimiento:

- llegar a los estados que se pueden alcanzar desde q y sólo pasan por arcos etiquetados con λ . Estos estados son los correspondientes a $\lambda\text{-clau}(q)$
- de todos los estados de $\lambda\text{-clau}(q)$ nos quedamos con aquellos estados r que tienen definida la transición $\Delta(r, a)$
- se calcula el conjunto de estados a los que se puede llegar desde estos estados r consumiendo en un paso el símbolo a y a este conjunto lo llamamos S
- calcular todos los estados alcanzables desde cualquier estado de S pasando sólo por arcos etiquetados con λ . Estos estados son los correspondientes a $\lambda\text{-clau}(S)$

El *AFND* que simule al *AFND*- λ debe poder alcanzar, para cada estado q y para cada símbolo a , los mismos estados a los que llega el *AFND*- λ a partir de q y leyendo el símbolo a . Por eso la nueva función de transición se calculará teniendo en cuenta el procedimiento anterior. Dado el autómatas $M_\lambda = (Q, V, \Delta, q_0, F)$ un *AFND* equivalente $M_N = (Q, V, \Delta', q_0, F')$ se obtiene definiendo:

- Si $\lambda\text{-clau}(q_0) \cap F \neq \emptyset$ entonces $F' = F \cup \{q_0\}$, en otro caso $F' = F$
- $\forall q \in Q, a \in V$ se define $\Delta'(q, a) = \lambda\text{-clau}(S)$,
donde $S = \{p \in Q \mid p \in \Delta(r, a) \wedge r \in \lambda\text{-clau}(q)\}$

Para demostrar que este método es correcto habría que probar que $w \in L(M_\lambda) \Leftrightarrow w \in L(M_N)$, $\forall w \in V^*$. Para $w = \lambda$ tenemos que $\lambda \in L(M_\lambda)$ sii $[(q_0, \lambda) \vdash_\lambda^* (p_F, \lambda) \wedge p_F \in F]$. Pero esto se cumple sii $[(p_F = q_0) \vee (p_F \in \lambda\text{-clau}(q_0))]$ sii $(q_0 \in F')$ sii $\lambda \in L(M_N)$, puesto que $(q_0, \lambda) \vdash_{M_N}^* (q_0, \lambda)$. Ahora se debe demostrar que $w \in L(M_\lambda) \Leftrightarrow w \in L(M_N)$ para todas las palabras de longitud mayor o igual que uno. El proceso a seguir es similar al del teorema 3.1 y necesitamos demostrar la hipótesis:

HIPÓTESIS : $(q, w) \vdash_{M_\lambda}^* (p, \lambda) \Leftrightarrow (q, w) \vdash_{M_N}^* (p, \lambda)$
(1)
(2)

Una vez demostrada la hipótesis, tomando $q = q_0$ se tiene claramente que $w \in L(M_\lambda) \Leftrightarrow w \in L(M_N)$. Luego $L(M_\lambda) = L(M_N)$, como queríamos demostrar. La demostración de la hipótesis se deja como ejercicio. ■

Ejemplo 3.7 Supongamos que queremos reconocer el lenguaje $L = \{0^i 1^j 2^k \mid i, j, k \geq 0\}$, o lo que es lo mismo, el lenguaje descrito por la expresión regular $0^* 1^* 2^*$. Podemos diseñar un autómata finito con λ -transiciones $M_\lambda = (\{q_0, q_1, q_2\}, \{0, 1, 2\}, \Delta, q_0, \{q_2\})$ y función Δ representada en la figura 3.7. El AFND equivalente es $M_N = (\{q_0, q_1, q_2\}, \{0, 1, 2\}, \Delta', q_0, \{q_0, q_2\})$, donde Δ' se define como:

$$\begin{array}{lll}
 \Delta'(q_0, 0) = \lambda\text{-clau}\{q_0\} = \{q_0, q_1, q_2\} & \Delta'(q_1, 0) = \emptyset & \Delta'(q_2, 0) = \emptyset \\
 \Delta'(q_0, 1) = \lambda\text{-clau}\{q_1\} = \{q_1, q_2\} & \Delta'(q_1, 1) = \lambda\text{-clau}\{q_1\} = \{q_1, q_2\} & \Delta'(q_2, 1) = \emptyset \\
 \Delta'(q_0, 2) = \lambda\text{-clau}\{q_2\} = \{q_2\} & \Delta'(q_1, 2) = \lambda\text{-clau}\{q_2\} = \{q_2\} & \Delta'(q_2, 2) = \lambda\text{-clau}\{q_2\} = \{q_2\}
 \end{array}$$

En la figura 3.7 aparecen los diagramas de los dos autómatas finitos. La palabra λ es aceptada por el AFND- λ por el siguiente cálculo: $(q_0, \lambda) \vdash_{M_\lambda} (q_1, \lambda) \vdash_{M_\lambda} (q_2, \lambda)$ y $q_2 \in F$. También es aceptada por el AFND ya que $(q_0, \lambda) \vdash_{M_N}^* (q_0, \lambda)$ y $q_0 \in F'$. Otra palabra aceptada es 012 ya que:

$$\begin{array}{l}
 (q_0, 012) \vdash_{M_\lambda} (q_0, 12) \vdash_{M_\lambda} (q_1, 12) \vdash_{M_\lambda} (q_1, 2) \vdash_{M_\lambda} (q_2, 2) \vdash_{M_\lambda} (q_2, \lambda) \text{ y } q_2 \in F \\
 (q_0, 012) \vdash_{M_N} (q_1, 12) \vdash_{M_N} (q_2, 2) \vdash_{M_N} (q_2, \lambda) \text{ y } q_2 \in F'
 \end{array}$$

Como vemos el cálculo es más corto (3 pasos) en el AFND que en el AFND- λ (5 pasos), ya que debido a las λ -transiciones el AFND- λ puede hacer “movimientos” sin consumir símbolos.

7. Autómatas finitos, expresiones regulares y gramáticas regulares

En el tema anterior estudiamos las expresiones regulares y vimos que pueden utilizarse para describir lenguajes regulares generados por gramáticas regulares o de tipo 3, según la clasificación de Chomsky. Ahora vamos a ver que si un lenguaje es aceptado por un autómata finito también se puede describir mediante una expresión regular y al contrario, todo lenguaje descrito por una expresión regular puede ser aceptado por un autómata finito.

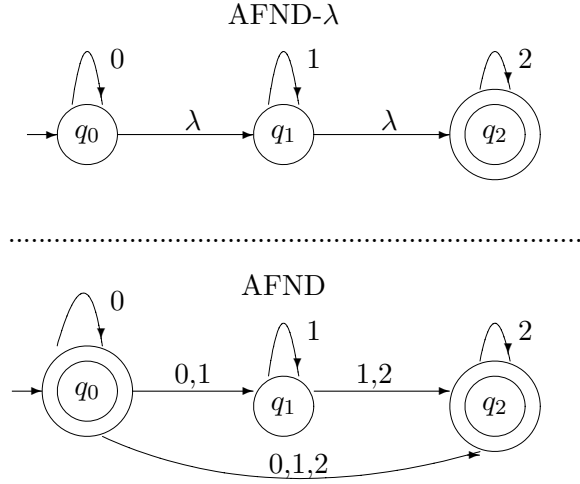


Figura 3.7: Ejemplo de paso de $AFND-\lambda$ a $AFND$

7.1. Teorema de Análisis de Kleene

Si L es un lenguaje aceptado por un autómata finito M entonces existe una expresión regular α tal que $L = L(M) = L(\alpha)$.

Podemos suponer que el autómata finito M no tiene λ -transiciones (si las tuviera ya sabemos que podemos encontrar autómata equivalente sin λ -transiciones). Sea $M = (Q, V, \Delta, q_0, F)$. A partir de este autómata podemos obtener un sistema de ecuaciones de expresiones regulares que llamaremos *ecuaciones características del autómata*. Estas ecuaciones se obtienen a partir del diagrama de transición del autómata del siguiente modo:

- A cada nodo q_i le corresponde una ecuación y cada estado se puede considerar como una incógnita de la ecuación.
- La ecuación para el estado q_i tiene en el primer miembro el estado q_i y el segundo miembro de la ecuación está formado por una suma de términos, de forma que por cada arco del diagrama de la forma $q_i \xrightarrow{a} q_j$ tenemos un término aq_j . Si el estado q_i es final, añadimos además el termino λ al segundo miembro.

Cada incógnita q_i del sistema de ecuaciones representa el conjunto de palabras que nos llevan del nodo q_i a un estado final, en el diagrama de transición. Por tanto, si resolvemos el sistema de las ecuaciones características del autómata tendremos soluciones de la forma $q_i = \alpha_i$, donde α_i es una expresión regular sobre el alfabeto V y como hemos dicho el lenguaje descrito por esta expresión regular es:

$$L(\alpha_i) = \{w \in V^* \mid (q_i, w) \vdash^* (q_F, \lambda), q_F \in F\} \quad (3.1)$$

El **método** que se propone para obtener una expresión regular α a partir de un AF es el siguiente:

1. Obtener las ecuaciones características del autómata;
2. Resolver el sistema de ecuaciones;
3. $\alpha \leftarrow$ solución para el estado inicial;

Para comprobar que este método es válido tendríamos que probar que se cumple 3.1 para toda solución $q_i = \alpha_i$ del sistema de ecuaciones, y en particular la solución para el estado inicial es la expresión regular correspondiente al autómata. Aunque no lo vamos a demostrar formalmente, por la forma de obtener las ecuaciones características del autómata y la validez del método de resolución de sistemas de ecuaciones de expresiones regulares vista en el tema anterior, podemos intuir que el método anterior es correcto. En realidad, no es necesario resolver todas las incógnitas, sólo necesitamos despejar la incógnita correspondiente al estado inicial.

Ejemplo 3.8 Consideremos de nuevo el autómata finito M de la figura 3.2. Las ecuaciones características correspondientes a este autómata son:

$$\begin{aligned} q_0 &= 0q_0 + 1q_1 \\ q_1 &= 0q_0 + 1q_2 + \lambda \\ q_2 &= 1q_1 + 0q_2 \end{aligned}$$

Comenzando por la última ecuación se tiene que $q_2 = 0^*1q_1$ y sustituyendo en la segunda ecuación queda

$$q_1 = 0q_0 + 10^*1q_1 + \lambda$$

de donde se obtiene que $q_1 = (10^*1)^*(0q_0 + \lambda)$ y este valor se sustituye en la primera ecuación

$$q_0 = 0q_0 + 1(10^*1)^*(0q_0 + \lambda) = 0q_0 + 1(10^*1)^*0q_0 + 1(10^*1)^*$$

Esta ecuación es fundamental y por el lema de Arden tiene como única solución

$$q_0 = (0 + 1(10^*1)^*0)^*1(10^*1)^*$$

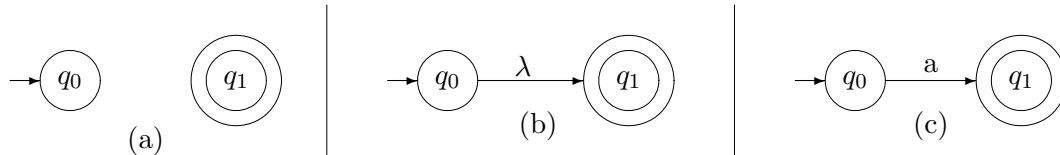
y por tanto $(0 + 1(10^*1)^*0)^*1(10^*1)^*$ es la expresión regular que describe el lenguaje $L(M)$.

7.2. Teorema de Síntesis de Kleene

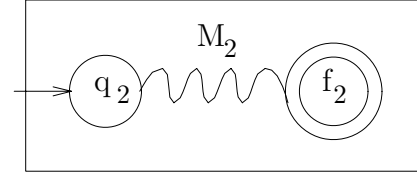
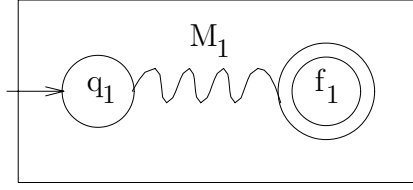
Si L es un lenguaje asociado a una expresión regular α entonces existe un autómata finito M tal que $L = L(\alpha) = L(M)$.

Vamos a demostrar por inducción sobre el número de operadores de α ($+$, $\cdot$, $*$) que existe un $AFND-\lambda$ M con un sólo estado final sin transiciones y distinto del estado inicial, de forma que $L(\alpha) = L(M)$.

Base.- (cero operadores) α puede ser: $\emptyset$, λ , a , donde $a \in V$. Los autómatas que aceptan el lenguaje vacío, el lenguaje $\{\lambda\}$ y el lenguaje $\{a\}$, son, por este orden, los siguientes: (a), (b) y (c)



Inducción.- (uno o más operadores en α). Supongamos que se cumple la hipótesis para expresiones regulares de menos de n operadores. Sean las expresiones regulares α_1 y α_2 donde $op(\alpha_1), op(\alpha_2) < n$. Entonces, por hipótesis existen dos autómatas finitos M_1 y M_2 tal que $L(M_1) = L(\alpha_1)$ y $L(M_2) = L(\alpha_2)$, donde $M_1 = (Q_1, V_1, \Delta_1, q_1, \{f_1\})$ y $M_2 = (Q_2, V_2, \Delta_2, q_2, \{f_2\})$ y podemos suponer sin pérdida de generalidad que $Q_1 \cap Q_2 = \emptyset$. Estos autómatas podemos representarlos esquemáticamente como:

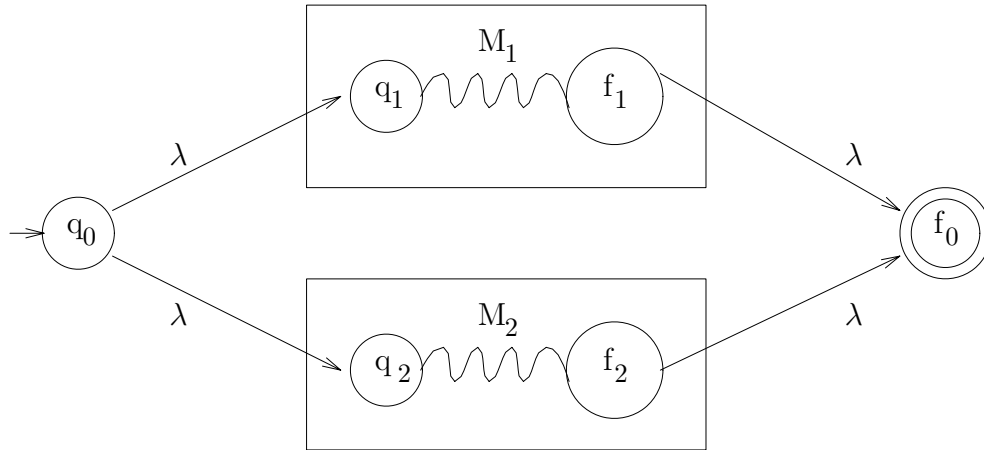


Supongamos que tenemos una expresión regular α con n operadores. Vamos a construir un automata M tal que $L(M) = L(\alpha)$ y para eso distinguimos tres casos correspondientes a las tres formas posibles de expresar α en función de otras expresiones regulares con menos de n operadores.

1. $\alpha = \alpha_1 + \alpha_2$ tal que $op(\alpha_1), op(\alpha_2) < n$. Los autómatas correspondientes a α_1 y α_2 son respectivamente M_1 y M_2 , como hemos dicho antes. A partir de M_1 y M_2 construimos otro autómata $M = (Q_1 \cup Q_2 \cup \{q_0, f_0\}, V_1 \cup V_2, \Delta, q_0, \{f_0\})$ donde Δ se define como:

- a) $\Delta(q_0, \lambda) = \{q_1, q_2\}$
- b) $\Delta(q, \sigma) = \Delta_1(q, \sigma), \quad \forall q \in Q_1 - \{f_1\}, \sigma \in V_1 \cup \{\lambda\}$
- c) $\Delta(q, \sigma) = \Delta_2(q, \sigma), \quad \forall q \in Q_2 - \{f_2\}, \sigma \in V_2 \cup \{\lambda\}$
- d) $\Delta(f_1, \lambda) = \Delta(f_2, \lambda) = \{f_0\}$

M se puede representar gráficamente del siguiente modo:



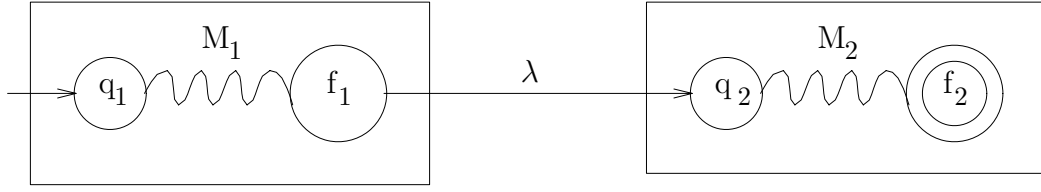
Cualquier camino de q_0 a f_0 debe pasar forzosamente a través del autómata M_1 o del autómata M_2 . Si una cadena w es aceptada por M , entonces debe ser aceptada también por M_1 o por M_2 .

Es decir, $L(M) = L(M_1) \cup L(M_2) =$
 $= L(\alpha_1) \cup L(\alpha_2)$, por hipótesis de inducción
 $= L(\alpha_1 + \alpha_2)$, por definición de lenguaje asociado a $\alpha_1 + \alpha_2$
 $= L(\alpha)$, como queríamos demostrar.

2. $\alpha = \alpha_1 \cdot \alpha_2$ tal que $op(\alpha_1), op(\alpha_2) < n$. A partir de M_1 y M_2 construimos otro autómata $M = (Q_1 \cup Q_2, V_1 \cup V_2, \Delta, q_1, \{f_2\})$ donde Δ se define como:

- a) $\Delta(q, \sigma) = \Delta_1(q, \sigma), \quad \forall q \in Q_1 - \{f_1\}, \sigma \in V_1 \cup \{\lambda\}$
- b) $\Delta(f_1, \lambda) = \{q_2\}$
- c) $\Delta(q, \sigma) = \Delta_2(q, \sigma), \quad \forall q \in Q_2, \sigma \in V_2 \cup \{\lambda\}$

M se puede representar esquemáticamente como:



Cualquier camino de q_1 a f_2 debe pasar forzosamente a través del autómata M_1 y del autómata M_2 . Si una cadena w es aceptada por M , entonces esa cadena se puede descomponer como $w = w_1.w_2$, de forma que w_1 debe ser aceptada por M_1 y w_2 por M_2 . Según esto, $L(M) = L(M_1) \cdot L(M_2) =$

$= L(\alpha_1) \cdot L(\alpha_2)$, por hipótesis de inducción

$= L(\alpha_1 \cdot \alpha_2)$, por definición de lenguaje asociado a $\alpha_1 \cdot \alpha_2$

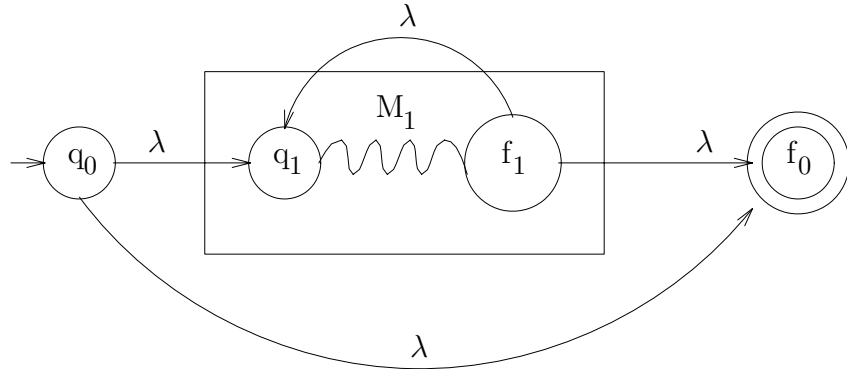
$= L(\alpha)$, como queríamos demostrar.

3. $\alpha = (\alpha_1)^*$ tal que $op(\alpha_1) = n - 1$. El autómata correspondiente a α_1 es M_1 , a partir del cual construimos otro autómata $M = (Q_1 \cup \{q_0, f_0\}, V_1, \Delta, q_0, \{f_0\})$ donde Δ se define como:

a) $\Delta(q_0, \lambda) = \Delta(f_1, \lambda) = \{q_1, f_0\}$

b) $\Delta(q, \sigma) = \Delta_1(q, \sigma)$, $\forall q \in Q_1 - \{f_1\}, \sigma \in V_1 \cup \{\lambda\}$

M se puede representar del siguiente modo:



Este autómata acepta cadenas de la forma $w = w_1 w_2 \cdots w_j$, donde $j \geq 0$ y cada subcadena w_i es aceptada por M_1 . Por tanto,

$$L(M) = \bigcup_{n=0}^{\infty} (L(M_1))^n =$$

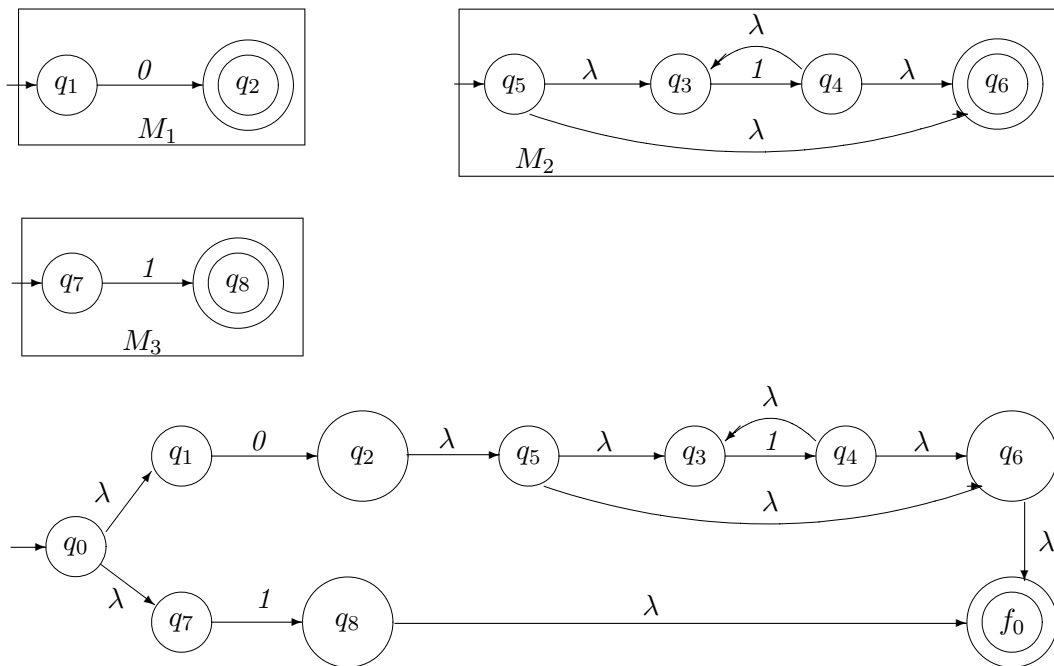
$$= \bigcup_{n=0}^{\infty} (L(\alpha_1))^n, \text{ por hipótesis de inducción}$$

$$= (L(\alpha_1))^*, \text{ por definición de clausura de un lenguaje}$$

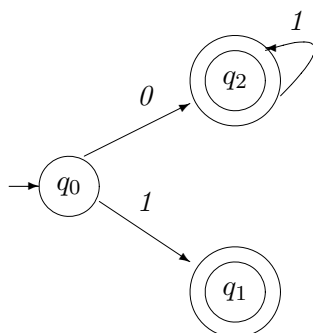
$$= L(\alpha_1^*), \text{ por definición de lenguaje asociado a } \alpha_1^*$$

$$= L(\alpha), \text{ como queríamos demostrar.}$$

Ejemplo 3.9 Siguiendo el método anterior, en la figura siguiente se ha construido un autómata para la expresión regular $01^* + 1$, donde M_1 representa el autómata para la expresión regular 0 , M_2 representa 1^* y M_3 la expresión regular 1 . En el autómata final se han integrado simultáneamente los autómatas para la concatenación (0 con 1^*) y la suma de expresiones regulares $01^* + 1$.



Este método de construcción de AFs para expresiones regulares está pensado para ser implementado de forma automática mediante un programa. Para nosotros podría haber sido más sencillo pensar directamente, por ejemplo, en el siguiente autómata:



7.3. Autómatas finitos y gramáticas regulares

Los autómatas finitos son mecanismos *reconocedores* de lenguajes y las gramáticas regulares son mecanismos *generadores* y vamos a ver que ambos tratan con la misma clases de lenguajes: los lenguajes regulares. Primero vamos a ver un teorema ($AF \longrightarrow GR$) cuya demostración nos proporciona un método para obtener una GR a partir de un AF y luego presentamos el teorema ($GR \longrightarrow AF$) para obtener un AF a partir de una GR .

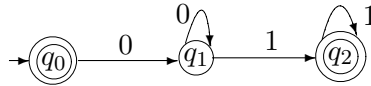
Teorema 3.3 ($AF \longrightarrow GR$) Si L es un lenguaje aceptado por una autómata finito M , entonces existe una gramática regular G tal que $L = L(M) = L(G)$.

Dem.- Podemos suponer sin pérdida de generalidad que $M = (Q, V, \Delta, q_0, F)$ es un AF que no tiene λ -transiciones. Podemos obtener la gramática $G = (Q, V, q_0, P)$ a partir del diagrama de transición del AF con el siguiente **método**:

1. Si tenemos el arco $q \xrightarrow{a} p$ entonces añadimos a P la regla $q \rightarrow ap$
2. Si $q_F \in F$ añadimos la regla $q_F \rightarrow \lambda$

Esta es la *parte constructiva* de la demostración. Falta la *parte inductiva* para probar que el método es válido: hay que demostrar que $w \in L(G) \Leftrightarrow w \in L(M)$. Lo dejamos para la sección de aplicaciones. ■

Ejemplo 3.10 Dado el siguiente AF, obtener la gramática correspondiente y la expresión regular.



La gramática que se obtiene es:

$$G = (\{q_0, q_1, q_2\}, \{0, 1\}, q_0, \{q_0 \rightarrow 0q_1 \mid \lambda, q_1 \rightarrow 0q_1 \mid 1q_2, q_2 \rightarrow 1q_2 \mid \lambda\})$$

Para obtener la expresión regular podemos obtener las ecuaciones características del autómata, o bien, obtener el sistema de ecuaciones para la gramática. En ambos casos se obtienen las mismas ecuaciones, que pasamos a resolver:

$$\begin{array}{lll} q_0 = 0q_1 + \lambda & q_0 = 0q_1 + \lambda & \\ q_1 = 0q_1 + 1q_2 & \dashrightarrow q_1 = 0q_1 + 1 \cdot 1^* & \dashrightarrow \\ q_2 = 1q_2 + \lambda & q_2 = 1^* & \end{array} \quad \boxed{q_0 = 00^*11^* + \lambda} \quad q_1 = 0^*11^*$$

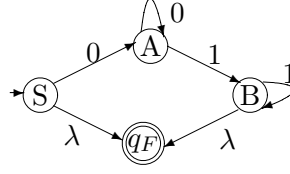
Teorema 3.4 ($GR \longrightarrow AF$) Si L es un lenguaje generado por una gramática regular G , entonces existe un autómata finito M tal que $L = L(G) = L(M)$.

Dem.- Podemos suponer sin pérdida de generalidad que $G = (V_N, V_T, S, P)$ es una gramática lineal derecha. Podemos obtener el diagrama del autómata finito $M = (V_N \cup \{q_F\}, V_T, \Delta, S, \{q_F\})$ a partir de la gramática con el siguiente **método**:

1. Si la regla $A \rightarrow aB \in P$ entonces añadimos el arco $A \xrightarrow{a} B$
2. Si la regla $A \rightarrow a \in P$ añadimos el arco $A \xrightarrow{a} q_F$
3. Si la regla $A \rightarrow \lambda \in P$ añadimos el arco $A \xrightarrow{\lambda} q_F$

Esta es la *parte constructiva* de la demostración. Falta la *parte inductiva* para probar que el método es válido. Como en el caso del teorema anterior, hay que demostrar que $w \in L(G) \Leftrightarrow w \in L(M)$. Lo dejamos como ejercicio. ■

Ejemplo 3.11 Dada la gramática con reglas: $P = \{S \rightarrow 0A \mid \lambda, A \rightarrow 0A \mid 1B, B \rightarrow 1B \mid \lambda\}$. El AF que reconoce el lenguaje generado por esta gramática es:



8. Minimización de un AFD

En esta sección vamos a ver cómo podemos obtener un AFD equivalente a uno dado que tenga el menor número de estados posibles. Para ello es necesario definir antes una relación de equivalencia en el conjunto de estados de un AFD.

Definición 3.1 Un estado q de un AFD es ACCESIBLE si $\exists x \in V^*$ tal que $(q_0, x) \vdash^* (q, \lambda)$. En otro caso el estado es INACCESIBLE.

Definición 3.2 Decimos que dos estados p y q de un AFD son EQUIVALENTES y lo notamos $p \approx q$, si cumplen:

$$\forall w \in V^* : \quad \text{si } [(q, w) \vdash^* (q', \lambda) \wedge (p, w) \vdash^* (p', \lambda)] \text{ entonces } [q' \in F \Leftrightarrow p' \in F]$$

Por otro lado, decimos que p y q son DISTINGUIBLES si no son equivalentes, o lo que es lo mismo, existe una cadena w tal que se produce una de las dos condiciones siguientes:

- $q' \in F$ y $p' \notin F$, o bien,
- $q' \notin F$ y $p' \in F$

Nota A partir de la definición anterior podemos hacer las siguientes observaciones:

1. Si tenemos dos estados $q \in F$ y $p \notin F$ entonces podemos asegurar que son distinguibles ya que tomando $w = \lambda$ se cumple la definición.
2. Si sabemos que dos estados (p_a, q_a) son distinguibles y se tiene que $p_a = \delta(p, a)$, $q_a = \delta(q, a)$ entonces podemos asegurar que (p, q) son distinguibles, ya que si una cadena w hace distinguibles a (p_a, q_a) entonces la cadena aw hace distinguibles a (p, q) .
3. $\approx$ define una RELACIÓN DE EQUIVALENCIA en el conjunto de estados del autómata y una forma de reducir el número de estados de un AFD será encontrar las clases de equivalencia en el conjunto de estados y a partir de ahí construir un autómata cuyos estados sean las clases de equivalencia.

Definición 3.3 Dado un AFD $M = (Q, V, \delta, q_0, F)$ se define el AUTÓMATA COCIENTE de M como $M_{\approx} = (Q', V, \delta', q'_0, F')$ donde:

$$\begin{aligned} Q' &= Q / \approx \\ \delta'([q], a) &= [\delta(q, a)] \\ q'_0 &= [q_0] \\ F' &= F / \approx \end{aligned}$$

Teorema

3.5 Dado un autómata finito determinista M , el autómata cociente $M_{\approx}$ es el autómata mínimo equivalente a M . Este autómata mínimo es único, salvo isomorfismos (renombramiento de estados).

Dem.- Primero tenemos que probar que M y $M_{\approx}$ son equivalentes. Pero para eso basta probar que $\forall w \in V^* : w \in L(M) \Leftrightarrow w \in L(M_{\approx})$. Para $w = \lambda$ está claro que $\lambda \in L(M) \Leftrightarrow q_0 \in F \Leftrightarrow [q_0] \in F' \Leftrightarrow \lambda \in L(M_{\approx})$. Cuando tenemos una palabra de longitud mayor que cero, por ejemplo $w = a_1 a_2 \dots a_n$ entonces $w \in L(M) \Leftrightarrow$ existe un cálculo donde $q_{i_n} \in F$ y:

$$(q_0, a_1 a_2 \dots a_n) \vdash_M (q_{i_1}, a_2 \dots a_n) \vdash_M \dots \vdash_M (q_{i_{n-1}}, a_n) \vdash_M (q_{i_n}, \lambda)$$

$$\Leftrightarrow q_{i_j} = \delta(q_{i_{j-1}}, a_j), \quad \forall 1 \leq j \leq n \text{ (por definición de la relación } \vdash)$$

$$\Leftrightarrow [q_{i_j}] = [\delta(q_{i_{j-1}}, a_j)], \quad \forall 1 \leq j \leq n \text{ (por definición de clase de equivalencia)}$$

$$\Leftrightarrow [q_{i_j}] = \delta'([q_{i_{j-1}}], a_j), \quad \forall 1 \leq j \leq n \text{ (por definición de } \delta')$$

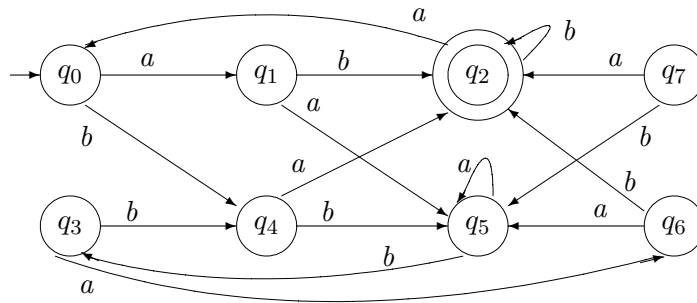
$$\Leftrightarrow ([q_0], a_1 a_2 \dots a_n) \vdash_{M_{\approx}} ([q_{i_1}], a_2 \dots a_n) \vdash_{M_{\approx}} \dots \vdash_{M_{\approx}} ([q_{i_{n-1}}], a_n) \vdash_{M_{\approx}} ([q_{i_n}], \lambda) \text{ donde } [q_{i_n}] \in F'$$

$$\Leftrightarrow w \in L(M_{\approx}), \text{ como queríamos demostrar.}$$

Ahora tenemos que probar no hay otro autómata equivalente a $M_{\approx}$ con menos estados que él. Supongamos que M' es equivalente a M y tiene menos estados que $M_{\approx}$. Entonces M' tiene que ser equivalente a $M_{\approx}$, lo cual implica que deben existir al menos dos estados distintos $[p]$ y $[q]$ en $Q / \approx$ que son equivalentes pero esto implica que $[p] = [q]$, lo cual es absurdo. Luego $M_{\approx}$ es único salvo isomorfismo (renombramiento de estados). ■

Una vez demostrada la existencia y unicidad del autómata mínimo mostramos en la figura 3.8 un **algoritmo que calcula el autómata cociente** o autómata mínimo. Aunque no vamos a demostrar formalmente la validez de este algoritmo, sería sencillo hacerlo a partir de la definición que hemos dado de la relación de equivalencia de estados, del autómata cociente y de los resultados expuestos en la nota anterior y el teorema 3.5. Aclaremos que cuando hablamos de $\text{PAR}(q_i, q_j)$ nos referimos a la posición de la tabla $T[i, j]$ cuando $i > j$ o a $T[j, i]$ cuando $i < j$. El hecho de utilizar una **tabla triangular** se justifica porque si $q_i \approx q_j$ entonces $q_j \approx q_i$, ya que la relación de equivalencia $\approx$ es simétrica. De esta forma se ahorra espacio de memoria. El **marcado recursivo** de $\text{lista}(q_i, q_j)$ significa que accedemos a todas posiciones correspondientes a las parejas de estados en $\text{lista}(q_i, q_j)$; marcamos estas celdas de la tabla y las celdas correspondientes a los pares de las listas asociadas a estas posiciones y así sucesivamente hasta que no se puedan marcar más.

Ejemplo 3.12 Vamos a calcular el autómata mínimo correspondiente al siguiente autómata:



Claramente se ve que el estado q_7 es inaccesible, por tanto, se puede eliminar este estado y sus transiciones. Tenemos que construir ahora una tabla triangular con filas desde q_1 hasta q_6 y columnas desde q_0 hasta q_5 . Marcamos la tabla que finalmente queda como sigue:

Entrada: Un AFD $M = (Q, V, \delta, q_0, F)$ con $Q = \{q_0, \dots, q_n\}$, $V = \{a_1, \dots, a_m\}$

Salida: AFD mínimo $M_{\approx} = (Q', V, \delta', q'_0, F')$

1. Eliminar estados inaccesibles de M ;
2. Construir tabla T con filas desde q_1 hasta q_n y columnas desde q_0 hasta q_{n-1} ;
3. Asociar a $PAR(q_i, q_j)$ una lista de parejas de estados $LISTA(q_i, q_j)$;
4. MARCAR ($PAR(q_i, q_j)$) donde un estado del par es final y el otro no;
5. for $i = 1$ to n
6. for $j = 0$ to $i - 1$
7. if $PAR(q_i, q_j)$ no marcado
8. for $k = 1$ hasta m
9. $q_r \leftarrow \delta(q_i, a_k)$; $q_s \leftarrow \delta(q_j, a_k)$;
10. if $PAR(q_r, q_s)$ marcado
11. MARCAR ($PAR(q_i, q_j)$)
- MARCAR-RECURSIVAMENTE ($LISTA(q_i, q_j)$);
- break;
12. else añadir a $LISTA(q_r, q_s)$ el $PAR(q_i, q_j)$;
13. end-if;
14. if $PAR(q_i, q_j)$ no marcado entonces $q_i \approx q_j$;
15. calcular Q', q'_0, δ', F' según la definición de autómata cociente;

Figura 3.8: Algoritmo de Minimización de un AFD

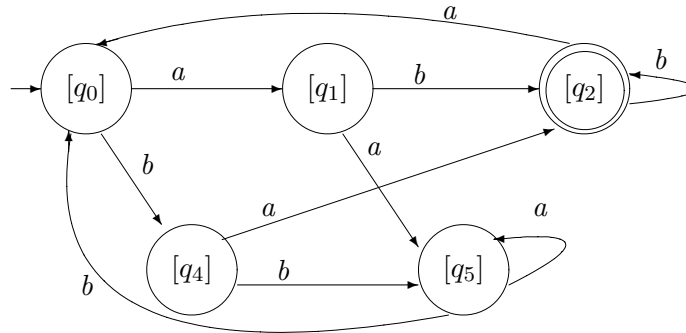
q_1	X					
q_2	$\boxed{X}$	$\boxed{X}$				
q_3		X	$\boxed{X}$			
q_4	X	X	$\boxed{X}$	X		
q_5	X	X	$\boxed{X}$	X	X	
q_6	X	(q_3, q_0)	$\boxed{X}$	X	X	X
	q_0	q_1	q_2	q_3	q_4	q_5

Las X recuadradas corresponden a las posiciones de la tabla que se marcan inicialmente (línea 4 del algoritmo). Al final quedan sin marcar $PAR(q_3, q_0)$ y $PAR(q_6, q_1)$ y por tanto, $q_0 \approx q_3$ y

$q_1 \approx q_6$. Y el autómata cociente (mínimo) es:

$$M_{\approx} = (\{[q_0], [q_1], [q_2], [q_4], [q_5]\}, \{0, 1\}, \delta', [q_0], \{[q_2]\})$$

cuyo diagrama de transición es:



El proceso de minimización de un *AFD* tiene gran **importancia práctica** ya que muchas aplicaciones de reconocimiento de patrones y de control se basan en la implementación de un *AFD*. Cuanto menos estados tenga el *AFD* más eficiente será la implementación.

9. Aplicaciones: análisis léxico

Una de las aplicaciones más importantes de los *AFs* es la construcción de analizadores léxicos. Como vimos en el tema 1, dentro del contexto general de un compilador, un *analizador léxico* (AL) tiene como principal función generar una lista ordenada de *tokens* a partir de los caracteres de entrada. Esos *tokens*, o componentes léxicos, serán usados por el analizador sintáctico para construir el correspondiente árbol sintáctico. Por otro lado, guarda información sobre algunos tokens, necesaria en el proceso de análisis y síntesis, en forma de atributos de esos componentes léxicos. Así pues, se debería considerar al analizador léxico como un módulo subordinado al analizador sintáctico (AS), tal y como se indica en el esquema de interacción entre ambos, que aparece en la figura 3.9.

El analizador léxico lee caracteres de entrada hasta que detecta que con el último carácter leído se puede formar un token, o un error en su caso, y comunica el evento correspondiente al analizador sintáctico. Si no hubo error, el AS procesa el token, y el AL no vuelve a entrar en juego hasta que el analizador sintáctico vuelva a necesitar otro *token* del flujo de entrada.

Otra de las funciones principales del AL es la detección y reparación de *errores léxicos*, aunque en este nivel la tarea de recuperación de errores no es muy sofisticada. Un error se produce cuando el AL detecta cualquier símbolo que es incapaz de reconocer y/o clasificar. Por ejemplo, la mayoría de versiones de PASCAL requieren que la expresión de un número en punto flotante comience con un 0: el *token* 0,5 pertenecería al lenguaje y ,5 no. Otro error que el analizador léxico podría detectar es el de exceder el número de caracteres máximo para un identificador.

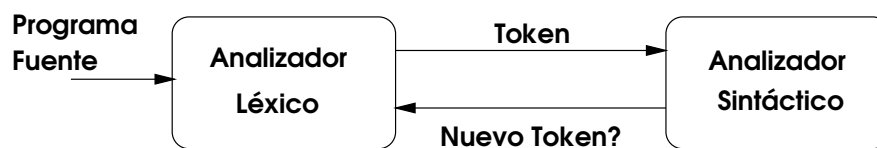


Figura 3.9: Esquema de interacción AL-AS

9.1. Especificación y reconocimiento de componentes léxicos

Los tokens se pueden *describir* mediante expresiones regulares y se pueden *reconocer* mediante autómatas finitos. Una expresión regular para un token, describe todos los *lexemas* que dicho token puede tener asociados. Los autómatas finitos se construyen a partir de las expresiones regulares que describen los tokens.

Para un buen reconocimiento léxico, los posibles tipos de *tokens* se deben diseñar con mucho cuidado. En general, un conjunto de cadenas a la entrada pueden corresponder al mismo componente léxico de salida. Cada conjunto de cadenas se va a definir mediante un patrón, asociado a un token determinado. En la siguiente tabla se dan ejemplos de varios tipos comunes de componentes léxicos, junto con lexemas ejemplo, y patrones que definen esos tokens.

<i>Token</i>	Lexemas ejemplo	Patrón no formal
const	const	const
if	if	if
relación	<, ≤, =, <>, ≥	< ó ≤ ó = ó <> ó ≥
identificador	pi, cuenta, D2	letra seguida de letras y dígitos
número	3.1416, 0, 6.02E23	cualquier cte. numérica
literal	“vaciado de memoria”	cualquier carácter entre “ y “ excepto “

Los componentes léxicos serán símbolos terminales de la gramática, y en la mayoría de lenguajes de programación se van a considerar como componentes léxicos las siguientes construcciones:

- **Palabras clave:** son cadenas que forman parte del lenguaje de programación en cuestión.
- **Operadores.**
- **Identificadores.**
- **Constantes** (reales, enteras y de tipo carácter).
- **Cadenas de caracteres.**
- **Signos de puntuación.**

Por simplicidad debe procurarse que los tokens sean sencillos y que los lexemas sean independientes. Aún así, podemos encontrarnos con *problemas a la hora de reconocer tokens*. A continuación analizamos algunos de ellos:

- A veces necesitamos leer uno o más caracteres extra de la entrada, denominados *caracteres de anticipación*, para decidir el código de token reconocido. Por ejemplo, si el lenguaje de programación admite los operadores “<” y “<=”, es necesario, una vez que se lee de la entrada el símbolo “<”, comprobar que si el siguiente es o no el símbolo “=”.
- Las *palabras clave* pueden estar reservadas o no. Si son *reservadas*, su significado está predefinido y el usuario no puede modificarlo usándolas como identificadores, por ejemplo. En este caso, el analizador léxico debe reconocerlas directamente (a través del autómata finito) o bien usando una tabla de palabras reservadas. Si las palabras clave *no* están *reservadas*, entonces el analizador léxico las reconoce como identificadores, y la tarea de distinguirlas de éstos queda relegada al analizador sintáctico. Por ejemplo, en PL/1 las palabras clave no son reservadas, y por lo tanto una sentencia de este tipo tiene sentido:

IF THEN THEN THEN = ELSE; ELSE ELSE = THEN;

- Los *comentarios* deben ser reconocidos y eliminados.

- Los *blancos* pueden actuar como delimitadores, en cuyo caso el AL debe eliminarlos sin más, o pueden no tener este papel. En este caso, además de eliminarlos, el AL debe agrupar lexemas. Por otro lado, en algunos lenguajes con formato de línea, como FORTRAN, se exige que ciertas construcciones aparezcan en posiciones fijas de la línea de entrada (por ejemplo, que se comience en la columna 7). Así, la alineación de un lexema puede ser importante para determinar si es correcto o no. En este caso, la definición de un token cuyo lexema está formado por seis blancos, podría facilitar esta labor. Hoy en día, se tiende a diseñar lenguajes de programación independientes del formato.
- La definición del token *EOF* (End Of File) puede facilitar el análisis sintáctico posterior, pues permitiría comprobar si después del final del programa aparecen más símbolos, o bien si el fichero termina antes de que termine la escritura de un programa completo.

9.2. Diseño de un analizador léxico

El proceso que se sigue para la implementación del analizador léxico puede resumirse en los siguientes **pasos**:

- Identificar los *tokens* del lenguaje, y definirlos utilizando ERs como herramientas expresivas de descripción.
- Obtener el AF correspondiente a las ERs que ha de reconocer el AL. Minimizar el número de estados del AF.
- Se ha de programar el autómata, simulando su ejecución con la técnica que se considere oportuna.
- Se ha de diseñar el interface entre la entrada estándar, de donde proviene el programa fuente, y el AL.
- Una vez que se ha conseguido simular el autómata, los tokens reconocidos van a ser utilizados por el analizador sintáctico. Por lo tanto se hace necesario el diseño de una interface adecuada, entre el AS y el autómata que simula el reconocedor. Normalmente se incluye el AL como una subrutina del AS, devolviéndole un token cada vez que el AS lo requiera, así como la información asociada a él. Con respecto a esto, es importante definir el TAD (Tipo Abstracto de Datos) que servirá como soporte a la *tabla de símbolos* necesaria en este proceso de interacción.
- Especificar qué tipo de manejo de errores va a seguir el AL.

Ejemplo 3.13 *Vamos a ver un ejemplo sencillo. El token que podemos llamar REAL representa las constantes reales en Fortran y se puede describir mediante la siguiente expresión regular*

$$\text{REAL} = D^+(\lambda + .) + (D^*.D^+)$$

donde D a su vez describe la expresión regular $D = (0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9)$. Para simplificar, vamos a suponer que tenemos un código fuente donde deben aparecer constantes reales separadas por blancos (denotado por B). El AFND- λ que reconoce una constante real según la expresión regular anterior e ignora los blancos que pueda haber antes, es el de la figura 3.10. La nueva función de transición Δ' del AFND equivalente es:

$$\begin{array}{ll} \Delta'(q_0, 0-9) = \{q_1, q_4\} & \Delta'(q_2, 0-9) = \{q_3\} \\ \Delta'(q_0, \cdot) = \{q_2\} & \Delta'(q_3, 0-9) = \{q_3\} \\ \Delta'(q_0, b) = \{q_0\} & \Delta'(q_4, 0-9) = \{q_4\} \\ \Delta'(q_1, \cdot) = \{q_2\} & \Delta'(q_4, \cdot) = \{q_5\} \\ \Delta'(q_1, 0-9) = \{q_1\} & \end{array}$$

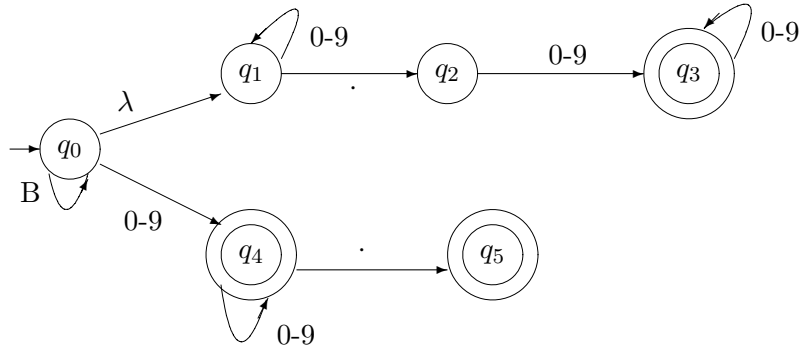


Figura 3.10: $AFND-\lambda$ para REAL

Finalmente se obtiene un AFD cuya función de transición δ se refleja en el diagrama de transición de la figura 3.11

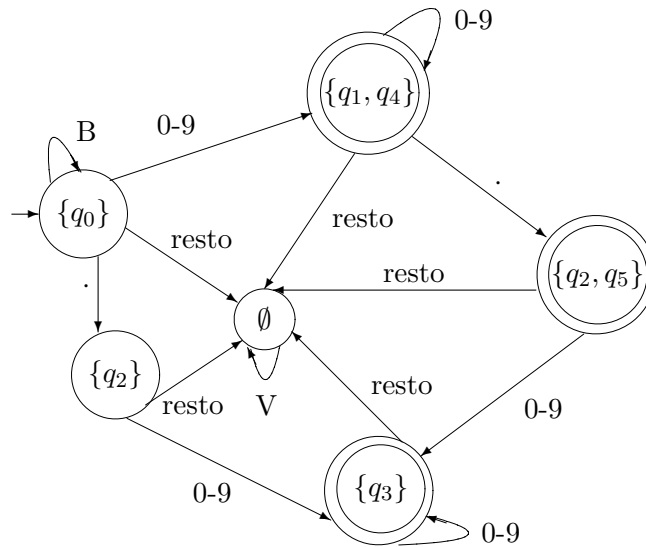


Figura 3.11: AFD para REAL

El AFD no es mínimo. Se puede comprobar que los estados $\{q_3\}$ y $\{q_2, q_5\}$ son equivalentes. Si minimizamos el AFD y renombramos los estados que quedan, tenemos el diagrama de transición del AFD mínimo en la figura 3.12.

Ahora **simulamos** el AFD mínimo a partir de su **diagrama de transición**, implementando en lenguaje C una función REAL1 que al ser llamada nos dice si se ha leído una constante real correcta o no. Se supone que lee los caracteres de la entrada estándar.

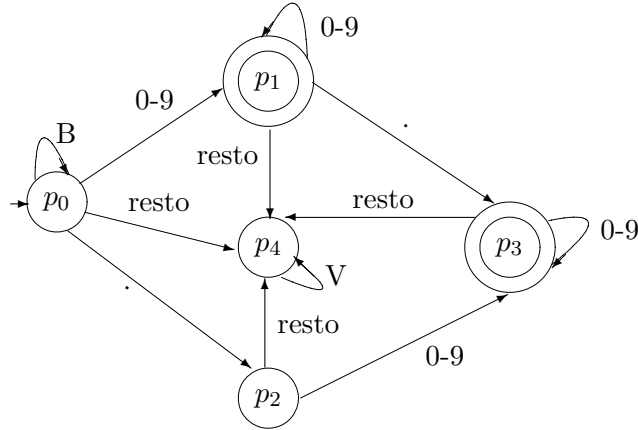


Figura 3.12: AFD mínimo para REAL

```

REAL() {
    int car;
    1. while (isspace(car=getchar())); /* se mantiene en p0 */
    2. ungetchar(car);
    3. if (isdigit(car=getchar())) { /* pasa a p1 */
    4.     for (car=getchar(); isdigit(car); car=getchar());
    5.     if (car=='.') { /* pasa a p3 */
    6.         for (car=getchar(); isdigit(car); car=getchar());
    7.         comprueba_fin(); } /* puede acabar en p3 */
    8.     else comprueba_fin();}
    9. else if (car=='V') /* pasa a p2 */
    10.    if (isdigit(car=getchar())) { /* pasa a p3 */
    11.        for (car=getchar(); isdigit(car); car=getchar());
    12.        comprueba_fin(); }
    13.    else puts(''constante real incorrecta'');
    14.    else puts(''constante real incorrecta'');}

    COMPRUEBA_FIN()
    15. if (car=='\ n') /* se ha leído toda la cte sin error*/
    16.    puts(''constante real correcta'');
    17. else puts(''constante real incorrecta'');}

```

Otra forma de simular el AFD es a partir de su **tabla de transición**. A continuación mostramos la tabla y en lugar de poner una columna para cada símbolo del alfabeto, utilizamos una de las columnas para todos los dígitos y otra para los demás caracteres que no sean un dígito, el punto decimal o un carácter blanco.

δ	b	$0-9$	$.$	$V - \{0-9, ., B\}$
p_0	p_0	p_1	p_2	p_4
p_1	p_4	p_1	p_3	p_4
p_2	p_4	p_3	p_4	p_4
p_3	p_4	p_3	p_4	p_4
p_4	p_4	p_4	p_4	p_4

A continuación mostramos el código para simular el AFD a partir de esta tabla, que se corresponde con la función REAL2.

```

#define ERROR 4
REAL2() {
    int estado, car, colum;
1. int tabla[5][4]={0,1,2,4,4,1,3,4,4,3,4,4,4,3,4,4,4,4,4,4};
    /* asignación por filas y usamos
    colum 0 → para carácter blanco
    colum 1 → para dígitos
    colum 2 → para el punto decimal
    colum 3 → para el resto de caracteres */
2. estado=0;
3. While ((car=getchar())!=\ n') {
4.     if (isspace(car)) colum=0;
5.     else if (isdigit(car)) colum=1;
6.         else if (car == '.') colum=2 ; else colum=3;
7.     estado=tabla[estado][colum];}
8. if ((estado==1) || (estado==3))
9.     puts('constante real correcta');
10. else puts('constante real incorrecta');}

```

9.3. Manejo de errores léxicos

No son muchos los errores que puede detectar un scanner; sin embargo, puede darse la situación de encontrar una cadena que no concuerde con ninguno de los patrones correspondientes a los *tokens* del lenguaje. ¿Qué hacer entonces?

El AL detectará un error cuando no exista transición válida en el autómata finito para el carácter de entrada. En este caso, el AL debe de informar del error, pero además, sería deseable que intentara recuperarse para seguir buscando otros errores. Aunque no existen métodos generales que funcionen bien en todos los casos, algunos de ellos se comportan de forma aceptable y son bastante eficientes. Por ejemplo:

- Recuperación en *modo pánico*: este tipo de estrategia es la más común. Consiste en ignorar caracteres extraños hasta encontrar un carácter válido para un nuevo token.
- Borrar un carácter extraño.
- Insertar un carácter que falta (e.g. reemplazar 2C por 2*C).
- Reemplazar un carácter incorrecto por otro correcto (e.g. reemplazar INTAGER por INTEGER si el lugar en donde aparece el primer lexema no es el indicado para un identificador).
- Intercambiar dos caracteres adyacentes.

Se suele utilizar el *modo pánico*, pues las otras técnicas, aunque más sofisticadas, también son más costosas de implementar. La recuperación de errores durante el AL puede producir otros en las siguientes fases. Por ejemplo, con el siguiente programa

```

var numero : integer;
begin
    num?ero:=10;
end

```

el compilador podría producir los siguientes mensajes de error:

- ERROR LÉXICO: carácter no reconocido (?)

- ERROR SEMÁNTICO: identificador no declarado (num)
- ERROR SINTÁCTICO: falta operador entre identificadores
- ERROR SEMÁNTICO: identificador no declarado (ero)

En otras ocasiones, sin embargo, la recuperación en modo pánico no conlleva efectos en otros ámbitos, como sería el caso del siguiente programa, en el se generaría un sólo aviso de error léxico (semejante al primero del ejemplo anterior):

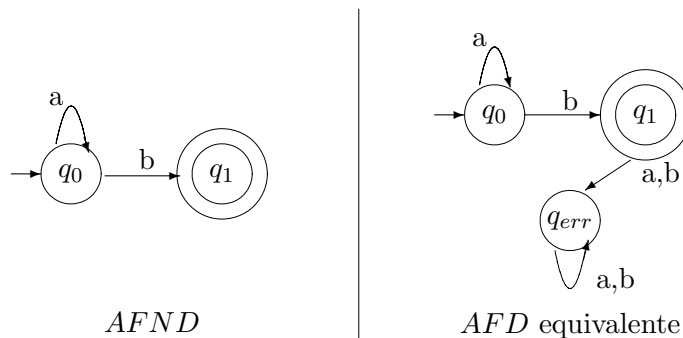
```
var i,j: integer;
begin
  i:=1;
  ?
  j:=2;
end
```

Finalmente, hay que tener en cuenta que existen errores que no son recuperables, como ocurriría por ejemplo, si se nos olvidara cerrar un comentario.

EJERCICIOS RESUELTOS

1. Dado un *AFND* que cumple $|\Delta(q, a)| \leq 1$ para todo estado y todo símbolo del alfabeto ¿Cómo se puede pasar de forma sencilla a un *AFD* equivalente?

Este autómata es “casi determinista”, lo único que puede ocurrir es que la función de transición no esté definida para alguna pareja (estado, símbolo). En este caso lo que hacemos es incluir un estado nuevo que actúa como *estado de error*, de forma que a él van a parar las transiciones no definidas y todos los arcos que salen del estado de error vuelven a él. De esta forma, si en un cálculo llegamos a una configuración con el estado de error, podemos estar seguros de que la palabra no será aceptada. Como ejemplo, podemos ver los siguientes diagramas de transición:



La palabra *abab* no es aceptada por el *AFND* ya que el único cálculo posible es el siguiente:

$$(q_0, abab) \vdash (q_0, bab) \vdash (q_1, ab) \vdash \text{(no puede seguir)}$$

Con el *AFD* tenemos el siguiente cálculo, donde se procesa toda la palabra, pero no acaba en configuración de aceptación:

$$(q_0, abab) \vdash (q_0, bab) \vdash (q_1, ab) \vdash (q_{err}, b) \vdash (q_{err}, \lambda)$$

-
2. ¿Cual es la expresión regular que describe el lenguaje aceptado por los autómatas anteriores?
-

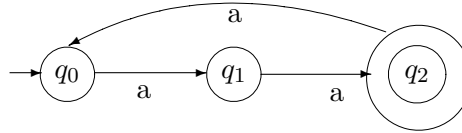
Parece sencillo ver a simple vista que $L(M) = \{a^n b \mid n \geq 0\}$, o lo que es lo mismo, $L(M) = L(a^*b)$. Para estar seguros podemos aplicar el método de las ecuaciones características. Se pueden obtener las ecuaciones para el *AFND* o el *AFD* indistintamente. Mejor obtener las correspondientes al *AFND* que tiene menos estados:

$$\begin{array}{l} q_0 = aq_0 + bq_1 \\ q_1 = \lambda \end{array} \quad \dashrightarrow \quad q_0 = aq_0 + b \quad \dashrightarrow \quad \boxed{q_0 = a^*b}$$

Si obtenemos las ecuaciones para el *AFD* el resultado es el mismo:

$$\begin{array}{l} q_0 = aq_0 + bq_1 \\ q_1 = (a+b)q_{err} + \lambda \\ q_{err} = (a+b)q_{err} \end{array} \quad \dashrightarrow \quad \begin{array}{l} q_0 = aq_0 + bq_1 \\ q_1 = (a+b) \cdot \emptyset + \lambda = \lambda \\ q_{err} = (a+b)^* \cdot \emptyset = \emptyset \end{array} \quad \dashrightarrow \quad q_0 = aq_0 + b \quad \dashrightarrow \quad \boxed{q_0 = a^*b}$$

3. Demostrar formalmente que $L(M) = \{(aaa)^n aa \mid n \geq 0\}$, donde M es el siguiente autómata finito:



Si probamos que $(q_i, a^m) \vdash^* (q_j, \lambda)$, donde $j = (m+i) \bmod 3$, entonces la palabra a^m será aceptada si $(i=0)$ y $(j=2)$, siendo $2 = m \bmod 3$. Luego debe ser $m = 3n+2$, $n \geq 0$, y en este caso $a^m = (aaa)^n aa$, que es lo que se indica en la descripción de $L(M)$ en el enunciado. Por tanto tenemos que probar la siguiente hipótesis:

$$\boxed{HIP. : \forall 0 \leq i \leq 2 \text{ se cumple } (q_i, a^m) \vdash^* (q_j, \lambda), \text{ donde } j = (m+i) \bmod 3}$$

Dem.- Lo demostramos por inducción sobre m .

Base.- ($m=0$) Entonces $j = i \bmod 3 = i$ y por ser la relación de cálculo $\vdash^*$ reflexiva se tiene trivialmente que $(q_i, \lambda) \vdash^* (q_i, \lambda)$.

Inducción.- Supongamos que se cumple la hipótesis $\forall m \leq k$ y vamos a demostrar que se cumple también para $m = k+1$. Como $m > 0$ podemos afirmar que:

$$(q_i, a^m) \vdash (q_{i'}, a^{m-1}) \vdash^* (q_j, \lambda)$$

para algún i', j . Por definición de la relación de cálculo y según el diagrama de transición, tenemos que $i' = (i+1) \bmod 3$. Por hipótesis de inducción podemos decir que $j = (m-1+i') \bmod 3$. Sustituyendo el valor de i' tenemos:

$$j = (m-1+(i+1) \bmod 3) \bmod 3$$

y por propiedades del módulo se tiene $j = (m+i) \bmod 3$, como queríamos demostrar. Por tanto el lenguaje acepta las palabras de la forma a^m donde $m = 3n+2$, $\forall n \geq 0$, condición que podemos expresar como $L(M) = \{(aaa)^n aa \mid n \geq 0\}$. ■

El ejemplo anterior nos muestra como un *AF* se puede usar para “contar” símbolos o

verificar si se cumplen algunas propiedades aritméticas, como por ejemplo, un lenguaje formado por palabras que tienen un número par de a 's. Pero los AF s son bastante limitados en este sentido. Ya veremos en el tema siguiente que no se pueden reconocer con AF s lenguajes más complicados como $L = \{a^n b^n \mid n \geq 0\}$, o el lenguaje $L = \{a^n \mid n \text{ es un cuadrado perfecto}\}$.

-
4. Probar la hipótesis del teorema 3.1 que muestra la validez del método para pasar de un $AFND$ M_N a un AFD M_D equivalente. La hipótesis es:

$$\boxed{\begin{array}{ll} HIP. : & (S, w) \vdash_{M_D}^* (S', \lambda) \Leftrightarrow S' = \left\{ p \in Q \mid (q, w) \vdash_{M_N}^* (p, \lambda) \wedge q \in S \right\} \\ & (1) \qquad \qquad \qquad (2) \end{array}}$$

Dem.- Vamos a demostrarlo por inducción sobre $|w|$.

Base.- ($|w| = 0$). Entonces $w = \lambda$. Por (1) tenemos que $(S, \lambda) \vdash_{M_D}^* (S', \lambda)$ sii (por definición de $\vdash_{M_D}^*$) $S = S'$. Pero esto es así ya que:

$$S' = \{p \in Q \mid (q, \lambda) \vdash_{M_N}^* (p, \lambda) \wedge q \in S\} = S$$

puesto que para que se de $(q, \lambda) \vdash_{M_N}^* (p, \lambda)$ debe ser $p = q$. Queda demostrado pues que (1) sii (2) para $|w| = 0$.

Inducción.- Supongamos que se cumple la hipótesis para palabras de longitud $< n$ y vamos a demostrar que se cumple también para $|w| = n$. Sea $w = az$ con $a \in V$ y $|z| = n - 1$. Por (1) y por definición de la relación de cálculo tenemos que:

$$\begin{array}{ll} (S, az) \vdash_{M_D} (S_1, z) & \vdash_{M_D}^* (S', \lambda) \\ (i) & (ii) \end{array}$$

$$\Leftrightarrow \left\{ \begin{array}{ll} (i) & S_1 = \delta(S, a) = \{r \in Q \mid r \in \Delta(q, a) \wedge q \in S\} \text{ por definición de } \delta \\ (ii) & S' = \left\{ p \in Q \mid (r, z) \vdash_{M_N}^* (p, \lambda) \wedge r \in S_1 \right\} \text{ por hipótesis} \end{array} \right\}$$

$$\Leftrightarrow (\text{por pertenecer } r \text{ a } S_1) S' = \left\{ p \in Q \mid (r, z) \vdash_{M_N}^* (p, \lambda) \wedge r \in \Delta(q, a) \wedge q \in S \right\} \Leftrightarrow$$

$$S' = \left\{ p \in Q \mid (r, z) \vdash_{M_N}^* (p, \lambda) \wedge (q, az) \vdash_{M_N} (r, z) \wedge q \in S \right\}$$

$$\Leftrightarrow (\text{por ser } \vdash_{M_N}^* \text{ la clausura reflexiva y transitiva de } \vdash_{M_N}):$$

$$S' = \{p \in Q \mid (q, w) \vdash_{M_N}^* (p, \lambda) \wedge q \in S\} \equiv (2), \text{ c.q.d.}$$

Probada ya la hipótesis y en particular para $S = \{q_0\}$ podemos decir que se cumple:

$$(\{q_0\}, w) \vdash_{M_D}^* (S', \lambda) \Leftrightarrow S' = \{p \in Q \mid (q_0, w) \vdash_{M_N}^* (p, \lambda)\}$$

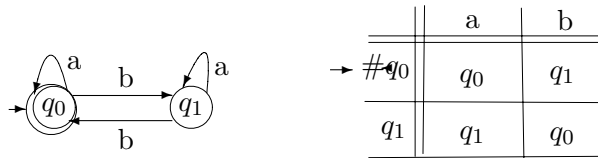
Luego tenemos que $w \in L(M_D) \Leftrightarrow S' \in F' \Leftrightarrow S' \cap F \neq \emptyset \Leftrightarrow \exists p_F \in F \wedge p_F \in S' \Leftrightarrow (q_0, w) \vdash_{M_N}^* (p_F, \lambda) \Leftrightarrow w \in L(M_N)$. Así que los dos autómatas aceptan los mismos lenguajes, luego el método es válido. ■

-
5. Escribir la tabla de transición y el diagrama de transición de un *AFD* tal que

$$L(M) = \{w \in \{a, b\}^* \mid b's(w) \text{ es par} \}$$

y encontrar un cálculo que acepte la palabra *aabba*

Podemos diseñar el *AFD* directamente, cuyas transiciones vienen dadas por:



El cálculo para *aabba* es:

$$(q_0, aabba) \vdash (q_0, abba) \vdash (q_0, bba) \vdash (q_1, ba) \vdash (q_0, a) \vdash (q_0, \lambda)$$

Luego $aabba \in L(M)$.

6. Dado el AFD de la figura 3.13 se puede comprobar que los estados q_0 y q_1 son equivalentes y lo mismo pasa con los estados q_2, q_3, q_4 . Por tanto el autómata mínimo es el que se refleja

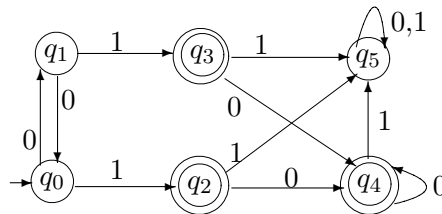
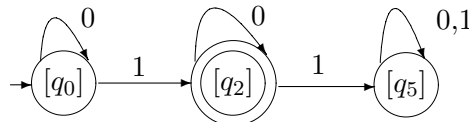


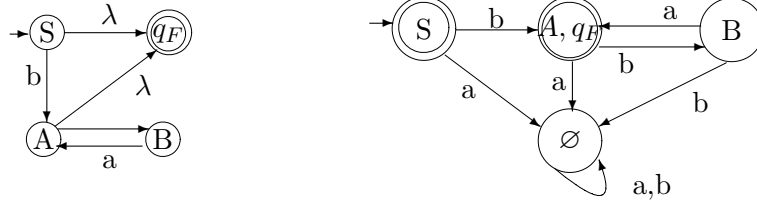
Figura 3.13: *AFD* que acepta $L(0^*10^*)$

en el siguiente diagrama:



-
7. Dada la gramática con reglas de producción $P = \{S \rightarrow bA \mid \lambda, A \rightarrow bB \mid \lambda, B \rightarrow aA\}$. Obtener un *AFD* que acepte el lenguaje generado por la gramática y una expresión regular que describa el lenguaje.
-

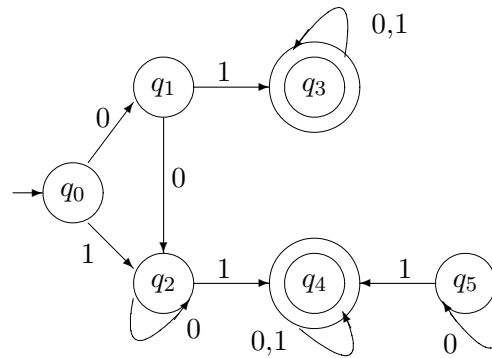
Este ejercicio podemos resolverlo de varias formas. Una de ellas sería obtener la expresión regular a partir de la gramática (según el método del tema 2) y luego obtener el autómata a partir de la expresión regular, según el teorema de síntesis de Kleene. En este caso es más sencillo obtener el autómata a partir de la gramática, según el método que hemos mostrado en este tema. A continuación mostramos los diagramas del $AFND-\lambda$ que se obtiene a partir de la gramática y el AFD equivalente:



Ahora podemos obtener el sistema de ecuaciones de expresiones regulares a partir de la gramática o del autómata. Lo hacemos a partir de la gramática, ya que se obtienen menos ecuaciones:

$$\begin{array}{l} S = bA + \lambda \\ A = bB + \lambda \\ B = aA \end{array} \quad \rightarrow \quad \begin{array}{l} S = bA + \lambda \\ A = baA + \lambda \end{array} \quad \rightarrow \quad \begin{array}{l} S = bA + \lambda \\ A = (ba)^* \cdot \lambda \end{array} \quad \rightarrow \quad \boxed{S = b(ba)^* + \lambda}$$

8. Minimizar el siguiente AFD



Aplicamos el algoritmo de minimización y lo primero que hacemos es eliminar el estado inaccesible q_5 . Construimos y marcamos la tabla triangular de la figura 3.14. Podemos observar que $q_1 \approx q_2$ y $q_3 \approx q_4$. Por tanto el autómata mínimo $M_{\approx}$ es el que se muestra en la figura 3.15.

9. Vamos a diseñar un analizador léxico para un sencillo lenguaje de programación denominado MICRO, cuyas especificaciones básicas son las siguientes:

- *Tipo de dato*: entero.
- No se declaran *variables*.
- Los *identificadores* empiezan por letra y se componen de letras, dígitos y símbolos de subrayado.

q_1	X			
q_2	X			
q_3	X	X	X	
q_4	X	X	X	(q_2, q_1)
	q_0	q_1	q_2	q_3

Figura 3.14: Marcado de la tabla

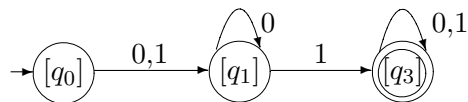


Figura 3.15: Autómata mínimo

- Solo hay *constantes* enteras.
- Los *comentarios* comienzan por `--` y terminan con EOL.
- *Sentencias*:
 - ID:=expresión;
 - read (lista de ID's);
 - write (lista de expresiones entre comas);
- *Palabras reservadas*: begin, end, read, write.
- Las *sentencias se separan* con `”;`.
- El *cuerpo del programa* está *delimitado* por begin/end.
- *Separadores de tokens*: espacios en blanco, tabuladores y new-line.

Expresiones regulares

ID : $L(L+D+{'-'})^*$

INTLITERAL : D^+

LPAREN : $($

RPAREN : $)$

SEMICOLON : $;$

COMMA : $,$

ASSIGNOP : $:=$

PLUSOP : $+$

```

MINUSOP : -
SCANEOF : EOF
BLANCOS : (' '+ '\n' + '\t')*
COMMENT : --C*EOL

```

Autómata finito

En la figura 3.16 aparece el autómata asociado a las expresiones regulares anteriores.

Implementación

El código que implemente el autómata finito anterior podría ser el que aparece a continuación, contenido en los archivos *lexico.h* y *lexico.c*.

```

/*
 * lexico.h
 */

typedef enum token_types { BEGIN, END, READ, WRITE, ID,
INTLITERAL, LPAREN, RPAREN, SEMICOLON, COMMA, ASSIGNOP, PLUSOP,
MINUSOP, SCANEOF } token;
extern token scanner(void);
extern char
token_buffer[]

/*
 * lexico.c
 */

#include "lexico.h"
#include <stdio.h>
#include <ctype.h>

void buffer_char(char c);
void clear_buffer(void); token
check_reserved(void); void lexical_error(char c); char
token_buffer[30];

token scanner(void) {    int in_char, c;
    clear_buffer();
    if (feof(stdin))
        return SCANEOF;
    while ((in_char = getchar()) != EOF) {
        if (isspace(in_char))
            continue; /*do nothing*/
        else if (isalpha(in_char)) {
            /*
             * ID ::= LETTER      | ID LETTER

```

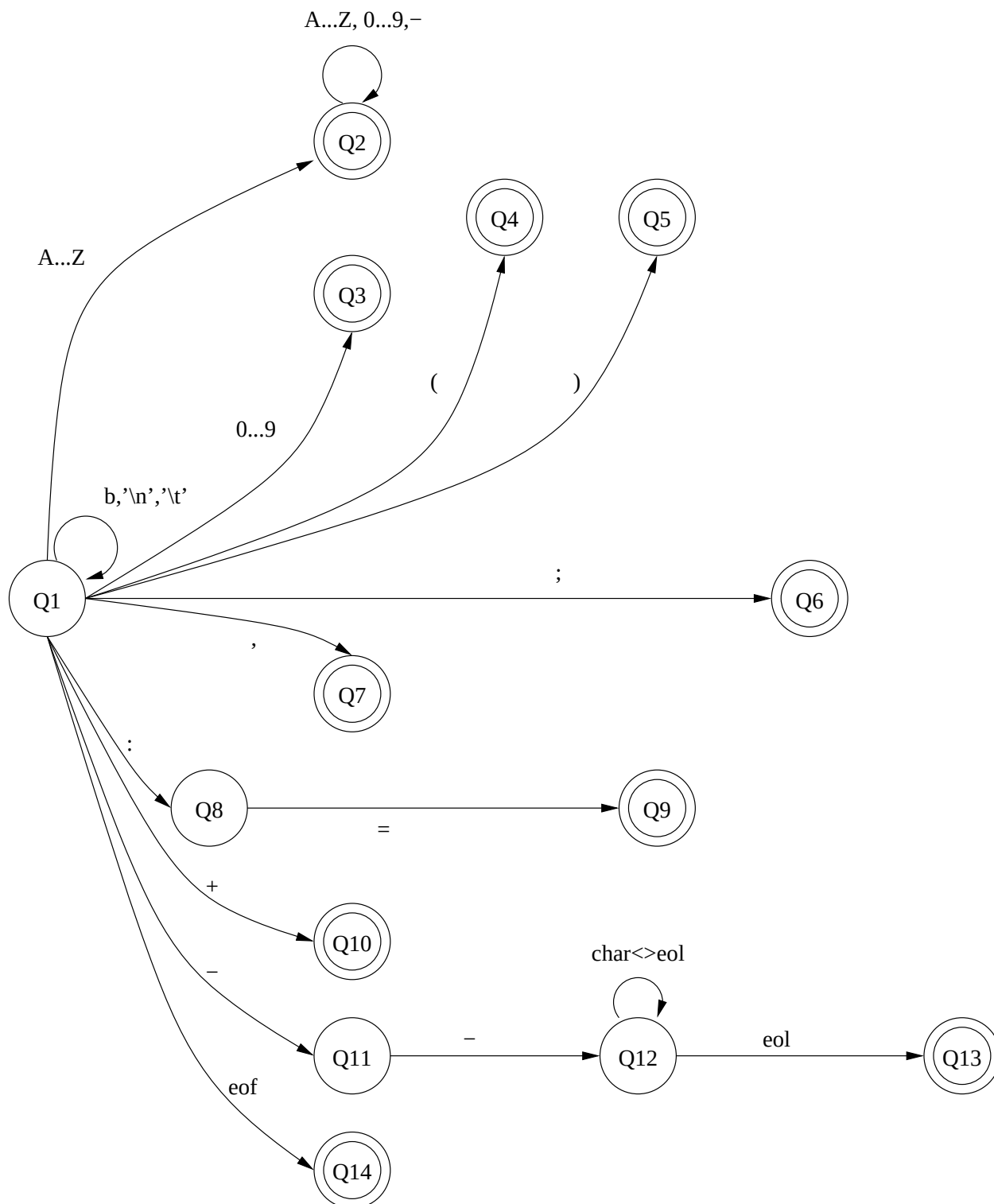


Figura 3.16: Autómata finito para reconocer las ER del lenguaje MICRO

```

        *                               | ID DIGIT
        *                               | ID UNDERSCORE
    */
    buffer_char(in_char);
    for(c=getchar(); isalnum(c) || c=='='; c=getchar())
        buffer_char(c);
    ungetc(c, stdin);
    return check_reserved();
} else if (isdigit(in_char)){
    /*
    * INTLITERAL ::= DIGIT
    *               | INTLITERAL DIGIT
    */
    buffer_char(in_char);
    for(c=getchar(); isdigit(c); c=getchar())
        buffer_char(c);
    ungetc(c, stdin);
    return INTLITERAL;
} else if (in_char == '(')
    return LPAREN;
else if (in_char == ')')
    return RPAREN;
else if (in_char == ';')
    return SEMICOLON;
else if (in_char == ',')
    return COMMA;
else if (in_char == '+')
    return PLUSOP;
else if (in_char == ':')
    /*looking for "!="*/
    c = getchar();
    if ( c=='=' )
        return ASSIGNOP;
    else {
        ungetc(c, stdin);
        lexical_error(in_char);}
} else if (in_char == '-') {
    /* is it --, comment start */
    c = getchar();
    if ( c == '-' ) {
        do
            in_char = getchar();
        while (in_char != '\n');
    } else {
        ungetc(c, stdin);
        return MINUSOP;}
} else lexical_error(in_char);
}
}

```

Las *palabras clave* son reservadas, pero no se reconocen en el autómata. La función *check_reserved()* comprueba si el lexema del identificador corresponde a una de ellas.

La *interfaz entre el programa fuente y el AL* se resuelve usando la entrada estándar (stdin) mediante las funciones *getchar()* para leer caracteres y *ungetc(c,stdin)* para devolver los caracteres de anticipación.

La *interfaz entre el AL y el AS* se realiza mediante la devolución de un token cada vez que el AS lo necesita (función *scanner()*). Por otro lado, a través de la variable global *token_buffer*, se pasan al AS los caracteres que forman parte de un identificador o constante entera. La función *buffer_char(c)* se encarga de añadir un carácter al buffer, y *clear_buffer()* borra todos sus caracteres.

El *manejo de errores* lo realiza la función *lexical_error(c)*, que informa de dicho error, continuando el análisis por el siguiente carácter.

EJERCICIOS PROPUESTOS

Se proponen los siguientes ejercicios para resolver en pizarra.

1. Construir AFDs para los siguientes lenguajes:

$$L_1 = \{w \in \{a,b\}^* \mid abab \text{ es subcadena de } w\}$$

$$L_2 = \{w \in \{a,b\}^* \mid \text{ni } aa \text{ ni } bb \text{ es subcadena de } w\}$$

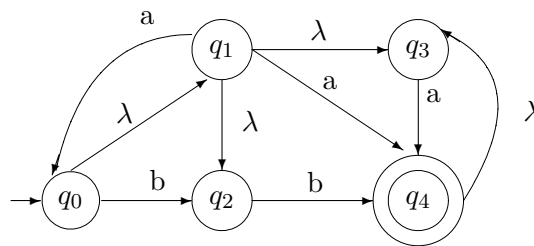
$$L_3 = \{w \in \{a,b\}^* \mid ab \text{ y } ba \text{ son subcadenas de } w\}$$

$$L_4 = \{w \in \{a,b\}^* \mid bbb \text{ no es subcadena de } w\}$$

2. Encontrar un AFD equivalente al AFND $M_N = (\{q_0, q_1\}, \{0, 1\}, \Delta, q_0, \{q_1\})$ donde

$$\Delta(q_0, 0) = \{q_0, q_1\}, \Delta(q_0, 1) = \{q_1\}, \Delta(q_1, 0) = \emptyset, \Delta(q_1, 1) = \{q_0, q_1\}$$

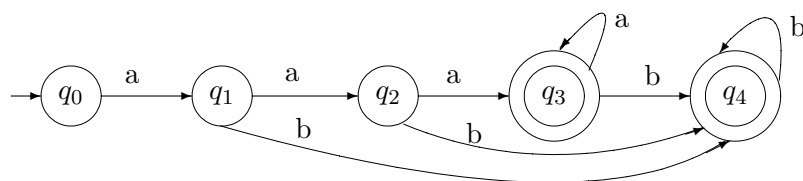
3. Obtener el AFD equivalente al siguiente autómata



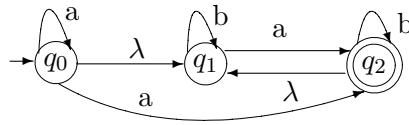
4. Construir un AFD y obtener la expresión regular asociada al lenguaje

$$L = \{w \in \{a,b\}^* \mid \text{cada } a \text{ en } w \text{ está precedida y seguida por una } b\}$$

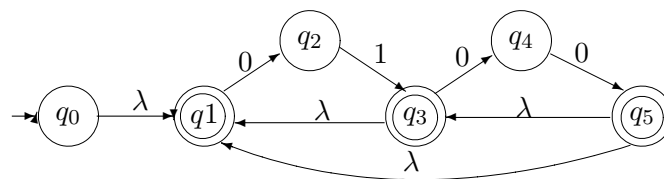
5. Obtener la expresión regular asociada al lenguaje aceptado por el autómata



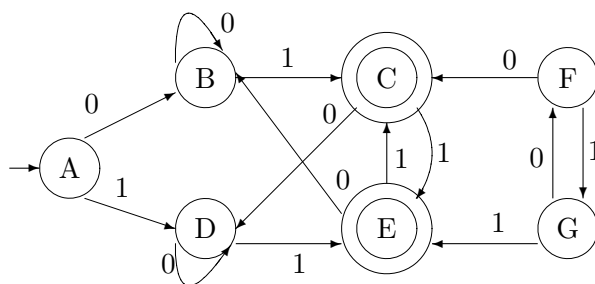
6. Demostrar la hipótesis del teorema 3.2 para probar que $w \in L(M_\lambda) \Leftrightarrow w \in L(M_N)$, para $|w| \geq 1$.
7. Dada la expresión regular $\alpha = a(bc)^*(b + bc) + a$: obtener un $AFND-\lambda$, un $AFND$ y un AFD que reconozcan $L(\alpha)$ y un cálculo para reconocer la palabra $abcb$ con cada uno de los autómatas.
8. Indicar un método para obtener un AF que reconozca L^R a partir del AF que reconoce L .
9. Obtener un AFD que reconozca el lenguaje formado por palabras sobre el alfabeto $V = \{0, 1\}$ que tienen un número par de ceros o un número de unos que sea múltiplo de tres. Resolver las ecuaciones características para obtener la expresión regular correspondiente.
10. Obtener la expresión regular que describe el lenguaje aceptado por el autómata:



11. Obtener un AFD que reconozca el lenguaje descrito por la expresión regular $(d(ab)^*)^* da(ba)^*$.
12. Dada la expresión regular $(ab)^*(ba)^* + aa^*$:
 - a) Obtener el AFD mínimo equivalente.
 - b) Aplicar el método $AFD\text{-mínimo} \longrightarrow GR$.
 - c) Obtener la gramática G con el método de las derivadas.
13. Dado el siguiente AF que llamaremos M_1 :



- a) Mostrar el cálculo por el que M_1 acepta la palabra 0101 y todas las configuraciones de parada posibles ¿Existen cálculos infinitos?
 - b) Modificar el diagrama de transición para obtener el $AFND$ equivalente M_2 .
 - c) ¿Cuántos pasos de cálculo se necesitan en M_2 para aceptar la palabra 0101? En este sentido ¿es más eficiente M_2 que M_1 ? ¿Por qué?
 - d) Mostrar la gramática G_2 a partir de M_2 . Obtener una gramática equivalente G_3 sin λ -producciones.
 - e) Obtener el AFD mínimo equivalente.
14. Dado el siguiente AFD :



- Minimizar el autómata.
- Obtener la gramática regular correspondiente.

15. Dada la expresión regular $0(0 + 1)^*0 + 10^*(1(0 + 1)^*0 + \lambda) + \lambda$:

- Obtener el *AFD* mínimo M .
- Obtener la *GR* que corresponde a la expresión regular por el método de las derivadas.

CUESTIONES BREVES

- Sea $M_{Q,V}$ el conjunto de todos los *AFDs* con conjunto de estados Q , alfabeto V , el mismo estado inicial y cada uno de ellos con un sólo estado final. ¿Cuál es el cardinal de $M_{Q,V}$?
- Sea M un *AF* (de cualquier tipo) con estado inicial q_0 . Si q_0 no es un estado final, ¿podemos asegurar que λ no pertenece a $L(M)$?
- Si dos *AFs* tienen distinta función de transición, ¿podemos asegurar que reconocen distintos lenguajes? ¿Y si los dos autómatas fueran deterministas?
- Dado un *AFND* M y una palabra $w \in L(M)$, ¿es posible que exista un cálculo para w cuya configuración de parada no sea de aceptación?
- Explicar cómo se obtendría el *AF* que reconoce $L(\alpha) \cup L(\beta)$, siendo α y β expresiones regulares.

NOTAS BIBLIOGRÁFICAS

- Respecto a los distintos tipos de autómatas que hemos visto, nosotros hemos diferenciado entre *AFND* y *AFND- λ* , siguiendo el ejemplo de autores como [Hop79] (capítulo 2, que recomendamos para las **secciones 1,2,3,4 y 7**).
- En cuanto a la definición de lenguaje aceptado por un autómata, hemos usado el concepto de configuración y relación de cálculo, como en el libro de [Lew81] (capítulo 2) (consultar para las **secciones 5 y 6**). Hemos preferido usar este enfoque porque consideramos más intuitivo el definir una relación de cálculo, que definir el lenguaje en términos más abstractos mediante el uso de la extensión de la función de transición (como hacen otros autores). Además los conceptos de configuración y cálculo y el uso que se hace de ellos para definir el lenguaje aceptado por una máquina abstracta, seguiremos usándolos para el resto de máquinas que estudiaremos en el curso.

3. Por último, en relación a los teoremas de Kleene, sólo existen diferencias sustanciales en la demostración del teorema de análisis. Nosotros hemos optado por el algoritmo de resolución de sistemas de ecuaciones de expresiones regulares (ver [Isa97], capítulo 3), que se aplica también en el tema anterior para obtener una expresión regular a partir de una gramática regular. Para el teorema de síntesis de Kleene se puede consultar el libro de [Hop79] (capítulo 2).